```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.ReelRiot.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S05_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.ReelRiot.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            // T011: the boot Save step also loads the meta progression - local
            // authority, the loaded state feeds the wire only (zero visuals).
            SaveWire.LoadOrDefault();
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            // T042-a: the boot locale decision lands here - editor faces
            // stay English (every frozen gate assertion holds), device
            // builds flip the rod copy to the zh table (the China-market
            // submission face).
            RodTexts.UseZh = RodTexts.DecideBootLocale();
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.ReelRiot.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// S1-T008: the ads service rides the channel suite (PlatformBootstrap) so
    /// ShowRewarded consumption is category-capped on the shared AdCap in every
    /// channel - the rescue pre-check and the consume can never disagree.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        /// &lt;summary&gt;Shared ad-cap instance (S1-T008): the rescue gate pre-checks
        /// the rescue category budget on the same policy the registered ads
        /// service consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
```

```
    public static readonly AdCapPolicy AdCap = new AdCapPolicy();
    /// &lt;summary&gt;Resolved channel suite (S1-T008). Non-null once RegisterAll
    /// ran; the suite's ads service is what ServiceLocator hands out.&lt;/summary&gt;
    public static PlatformSuite Suite { get; private set; }
    public static void RegisterAll()
    {
        if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
        ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        // S1-T008: channel suite wiring. PlatformBootstrap gates SY_WECHAT /
        // SY_DOUYIN builds to the real channel suites; editor/batchmode stays
        // mock. Ads consume from the shared AdCap in every channel so the
        // pre-check/consume same-instance law survives the real-suite switch.
        Suite = PlatformBootstrap.Create(AdCap);
        ServiceLocator.Register&lt;PlatformSuite&gt;(Suite);
        ServiceLocator.Register&lt;IAdsService&gt;(Suite.Ads);
        Logger.Info("BOOT", "services registered (suite=" + PlatformBootstrap.SuiteIdFor(Suite.Channel) + ")");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
```

```
    }
    public sealed class MockLocaleService : ILocaleService
    {
        public void SetLang(string lang) { }
        /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        public FontSet GetFontSet()
            =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
    }
}
// BandFx.cs
// BandFx.cs - the chosen-lane highlight + odds face (G05-T030 slice 2).
// The lane pick is agency over WHAT the next cast fishes, and the face
// answers on the same beat: the pick logs on the splash tick, so the glow
// is live from beat one (alpha rides the HintFx breathing envelope - age 0
// reads exactly AlphaFloor + AlphaSpan * 0.5, single-source law). One accent
// overlay spans the chosen half of the pond (shallow = the top half, deep =
// the bottom half - the same halves the density bands tile), and six odds
// slots render the authored per-band category table straight off BandSpecs:
// slot color = the codex palette hue of its table entry, slot size follows
// the lane's weight family (heavy hauls read bigger), slot brightness
// follows the codex seen ledger - the odds face never invents numbers, it
// renders the authored tables. Pure view: reads the model, never writes it;
// a null root or a classic None cast leaves the rig dark (null-safe lane
// law). New-file law: WaterView and HintFx stay byte-stable.
using System;
using UnityEngine;
namespace Sy.ReelRiot
{
    public sealed class BandFx
    {
        public const int OddsSlots = 6;             // the authored table size both lanes share
        public const float LaneHalf = WaterBodyData.Height * 0.5f;   // one half of the pond per lane
        public const float OddsSlotGapX = 1.35f;     // matches the codex strip squint spacing
        public const float ShallowSlotScale = 0.70f; // light fun props read small
        public const float DeepSlotScale = 1.05f;    // heavy hauls read big - the squint weight law
        public const string GlowName = "band_lane_glow";
        public const string SlotNamePrefix = "band_odds_slot_";
        static readonly Color AccentColor = Hex(0xE8833A);   // buoy warm orange - the pitch accent (HintFx same source)
        readonly Transform root;
        static Sprite circle;
        public BandFx(Transform root) { this.root = root; }
        /// &lt;summary&gt;Builds the lane rig: one half-pond glow rect plus the six
        /// odds slots. Returns the number of objects created (0 when rootless).&lt;/summary&gt;
        public int Rebuild()
        {
```

```
        Clear();
        int built = 0;
        built += Make(GlowName, AccentColor, Vector3.zero, Vector3.zero, 5) != null ? 1 : 0;
        for (int i = 0; i &lt; OddsSlots; i++)
            built += Make(SlotNamePrefix + i, AccentColor, Vector3.zero, Vector3.one, 6) != null ? 1 : 0;
        return built;
    }
    /// &lt;summary&gt;Per-step face on an explicit clock (hosts may blend; gates
    /// probe the same way): lights the glow on the chosen half and renders
    /// the lane's odds slots from the pick stamp. Dark when no lane pick
    /// lives - the classic None path never lights the face.&lt;/summary&gt;
    public void ApplyAt(BandChoice band, double pickAt, AbsurdCodex codex, double now)
    {
        var glow = FindChild(GlowName);
        if (glow == null) return;
        bool live = band == BandChoice.Shallow || band == BandChoice.Deep;
        glow.SetActive(live);
        for (int i = 0; i &lt; OddsSlots; i++)
        {
            var slot = FindChild(SlotNamePrefix + i);
            if (slot != null) slot.SetActive(live);
        }
        if (!live) return;
        double age = now - pickAt;
        glow.transform.localPosition = new Vector3(0f, LaneCenterY(band), 0f);
        glow.transform.localScale = new Vector3(WaterBodyData.Width, LaneHalf, 1f);
        var gsr = glow.GetComponent&lt;SpriteRenderer&gt;();
        if (gsr != null)
        {
            Color c = AccentColor;
            c.a = HintFx.AlphaFor(age);
            gsr.color = c;
        }
        for (int i = 0; i &lt; OddsSlots; i++)
        {
            var slot = FindChild(SlotNamePrefix + i);
            if (slot == null) continue;
            slot.transform.localPosition = OddsSlotPos(band, i);
            float s = band == BandChoice.Deep ? DeepSlotScale : ShallowSlotScale;
            slot.transform.localScale = new Vector3(s, s, 1f);
            var ssr = slot.GetComponent&lt;SpriteRenderer&gt;();
            if (ssr == null) continue;
            Color sc = OddsColor(band, i);
            int idx = BandSpecs.ItemIndexFor(band, i);
            bool seen = codex != null &amp;&amp; idx &gt;= 0 &amp;&amp; idx &lt; codex.Seen.Length &amp;&amp; codex.
            sc.a = seen ? 1f : WaterView.UnseenAlpha;
            ssr.color = sc;
        }
    }
    /// &lt;summary&gt;Host one-liner (SignalFx host law): the whole face derives
```

```
/// from the live model - the chosen lane, the pick stamp and the codex
/// ledger. Pure read: the model never learns the face exists.&lt;/summary&gt;
public void ApplyAt(CastModel m, double now)
{
    BandChoice band = m != null ? m.SelectedBand : BandChoice.None;
    double pickAt = m != null ? m.TimeOf("band_pick") : -1.0;
    ApplyAt(band, pickAt, m != null ? m.Codex : null, now);
}
/// &lt;summary&gt;Lane geometry law (single source): shallow tiles the top
/// half of the pond, deep the bottom half - the halves the six density
/// bands tile, so the glow never invents its own coordinate frame.&lt;/summary&gt;
public static float LaneCenterY(BandChoice band)
{
    return band == BandChoice.Deep ? LaneHalf * 0.5f : -LaneHalf * 0.5f;
}
/// &lt;summary&gt;Odds slot i rides the lane center line, spread like the
/// codex strip - the check proves the built slots sit on this exact
/// formula (no cross-file re-derivation can drift).&lt;/summary&gt;
public static Vector3 OddsSlotPos(BandChoice band, int i)
{
    return new Vector3((i - (OddsSlots - 1) * 0.5f) * OddsSlotGapX, LaneCenterY(band), 0f);
}
/// &lt;summary&gt;Slot color law: the codex palette hue of the lane's table
/// entry - the odds face and the codex strip share one palette, so a
/// lit slot reads as its category at a squint.&lt;/summary&gt;
public static Color OddsColor(BandChoice band, int i)
{
    return WaterView.ItemColor(BandSpecs.ItemIndexFor(band, i));
}
/// &lt;summary&gt;Looks up a built child object by name. Null when absent.&lt;/summary&gt;
public GameObject FindChild(string childName)
{
    if (root == null) return null;
    Transform t = root.Find(childName);
    return t == null ? null : t.gameObject;
}
public void Clear()
{
    if (root == null) return;
    for (int i = root.childCount - 1; i &gt;= 0; i--) WaterView.SafeDestroy(root.GetChild(i).gameObject);
}
static Sprite Circle
{
    get { if (circle == null) circle = MakeSprite(32, true); return circle; }
}
static Sprite MakeSprite(int size, bool round)
{
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    float r = size * 0.5f;
    for (int y = 0; y &lt; size; y++)
```

```
            {
                for (int x = 0; x &lt; size; x++)
                {
                    float dx = x + 0.5f - r;
                    float dy = y + 0.5f - r;
                    bool inside = !round || dx * dx + dy * dy &lt;= r * r;
                    tex.SetPixel(x, y, inside ? Color.white : Color.clear);
                }
            }
            tex.Apply();
            return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
        }
        GameObject Make(string name, Color color, Vector3 localPos, Vector3 scale, int sortingOrder)
        {
            if (root == null) return null;
            var go = new GameObject(name);
            go.transform.SetParent(root, false);
            go.transform.localPosition = localPos;
            go.transform.localScale = scale;
            var sr = go.AddComponent&lt;SpriteRenderer&gt;();
            sr.sprite = Circle;
            sr.color = color;
            sr.sortingOrder = sortingOrder;
            return go;
        }
        static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
    }
}
// CameraFit.cs
// CameraFit.cs - deterministic device-adaptation contract (G05-T027, device matrix).
// Red line 10 portrait lock pairs with this fit: the game column is a frozen
// DESIGN FRAME - the full fishing column from the LiveOps landing row up to the
// rod-tip glow - that must stay fully visible inside the SAFE area on every
// device. The pond itself (40x30) is a decorative school field: its edges may
// letterbox away, the column never clips. Apply computes an orthographic camera
// mapping the design frame into the safe rect; numbers come in as data so
// headless gates drive tier cards (1080x1920 / 720x1280) deterministically.
// CameraFitHost (wired into S05_Main by ProjectSetup) is the runtime face.
using UnityEngine;
namespace Sy.ReelRiot
{
    public static class CameraFit
    {
        /// &lt;summary&gt;Design frame: the must-see column. Bottom = LiveOps landing
        /// row (SlotPos past the base strip, -17.65 with slot half); top = the
        /// tension glow crown over the rod tip (21.15); width = the 12-slot codex
        /// strip span (+-7.875 with slot half) plus margin.&lt;/summary&gt;
```

```
public static readonly Rect DesignFrame = new Rect(-8.05f, -17.85f, 16.1f, 39.15f);
/// &lt;summary&gt;Pixel-density floor per tier (px per world unit, safe-rect
/// basis): a 0.9-unit codex slot must stay readable at 720p low tier.&lt;/summary&gt;
public const float PxPerWuFloor1080 = 42f;
public const float PxPerWuFloor720 = 27f;
/// &lt;summary&gt;Result of one fit: the safe rect in world space plus the
/// pixels-per-world-unit the fit produced (legibility face).&lt;/summary&gt;
public struct FitResult
{
    public Rect SafeWorld;
    public float PxPerWu;
}
/// &lt;summary&gt;Fits the camera so the design frame sits fully inside the
/// safe viewport rect. safeNv is normalized (x, y, w, h in 0..1).&lt;/summary&gt;
public static FitResult Apply(Camera cam, int screenW, int screenH, Rect safeNv)
{
    float safeWpx = screenW * safeNv.width;
    float safeHpx = screenH * safeNv.height;
    float safeAspect = safeWpx / safeHpx;
    float designAspect = DesignFrame.width / DesignFrame.height;
    // world units per pixel: bind the wider-relative axis, letterbox the other
    float wpp = designAspect &gt;= safeAspect
        ? DesignFrame.width / safeWpx
        : DesignFrame.height / safeHpx;
    cam.orthographic = true;
    cam.orthographicSize = screenH * 0.5f * wpp;
    float visW = screenW * wpp;
    float visH = screenH * wpp;
    // center the design frame on the safe rect's center
    float cu = safeNv.x + safeNv.width * 0.5f;
    float cv = safeNv.y + safeNv.height * 0.5f;
    float cx = DesignFrame.center.x - (cu - 0.5f) * visW;
    float cy = DesignFrame.center.y - (cv - 0.5f) * visH;
    cam.transform.position = new Vector3(cx, cy, cam.transform.position.z);
    float sx = cx + (safeNv.xMin - 0.5f) * visW;
    float sy = cy + (safeNv.yMin - 0.5f) * visH;
    return new FitResult
    {
        SafeWorld = new Rect(sx, sy, safeWpx * wpp, safeHpx * wpp),
        PxPerWu = 1f / wpp,
    };
}
/// &lt;summary&gt;Runtime face: fit the scene's main camera from the real
/// screen and its safe area (notch/corners compensate by letterbox).
/// Null-safe and skips on degenerate screen dims so headless gates that
/// build the composition without a camera never break.&lt;/summary&gt;
public static void ApplyToMainCamera()
{
    Camera cam = Camera.main;
    if (cam == null) return;
```

```
            int w = Screen.width, h = Screen.height;
            if (w &lt;= 0 || h &lt;= 0) return;
            Rect safe = Screen.safeArea;
            Rect safeNv = new Rect(safe.x / w, safe.y / h, safe.width / w, safe.height / h);
            Apply(cam, w, h, safeNv);
        }
    }
}
// CameraFitHost.cs
// CameraFitHost.cs - S05_Main runtime hook for the device-adaptation contract
// (G05-T027). G05's scene shell has no game host yet (the model-driven gates
// build compositions directly), so this tiny component is the scene-side face
// that applies CameraFit to the main camera on play start. Idempotent by
// construction: Apply is a pure function of the screen, so re-entering the
// scene refits the same numbers.
using UnityEngine;
namespace Sy.ReelRiot
{
    public sealed class CameraFitHost : MonoBehaviour
    {
        void Start()
        {
            CameraFit.ApplyToMainCamera();
        }
    }
}
// CastModel.cs
// CastModel.cs - cast-line core feel for ReelRiot (G05-T005/T006).
// E4 graybox Cast() timeline ported to a frame-stepped engine model:
// splash -&gt; lure flight -&gt; sink -&gt; bite with the same-tick feedback triple
// (ripple + sfx + alert) -&gt; tug-of-war pulses -&gt; land the absurd catch
// (delight triple + catalog entry + photo-card share moment) or deslot
// (+ same-tick rescue offer). Every E4-frozen duration (0.20 flight,
// 0.65/0.85/1.05 sink, 0.50/0.55/0.70 pulse) is an exact multiple of the
// 0.05s step, so frame-stepped event times equal the instantaneous graybox
// times exactly: bite 0.85/1.05/1.25s, resolve 2.35/3.25/4.75s by weight
// class. "Three seconds a cast" law: the bite always lands within 3s of the
// splash - the player never waits for action. Zero RNG: tug outcomes ride
// the injected tug plan, resolution stays pure timing skill.
// T006 struggle window (E3#2): every fight spans bite -&gt; resolve as a
// FightRecord (struggle window 1.50/2.20/3.50s by class - the 1.50s floor is
// the E4 min_struggle baseline) and every pulse is a TensionBeat: the fish
// yanks the rod on a deterministic alternating L/R ladder with the
// weight-class amplitude - the tension beat is what the rod view renders.
using System;
using System.Collections.Generic;
namespace Sy.ReelRiot
{
    public enum CastPhase { Idle, Flight, Sink, Fight, Resolved }
    public enum CastOutcome { None, Caught, Deslot }
```

```
/// &lt;summary&gt;One logged beat: logical time, event type, detail.&lt;/summary&gt;
public sealed class CastEvent
{
    public double t;
    public string type;
    public string detail;
    public CastEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
}
/// &lt;summary&gt;One tension beat of the tug-of-war (G05-T006): the fish yanks
/// the rod - it pulls you, not the other way around (signature move).
/// Direction alternates every beat on a deterministic L/R ladder, the
/// amplitude rides the weight-class pulse duration, so the chaos feel
/// never touches RNG.&lt;/summary&gt;
public sealed class TensionBeat
{
    public double t;          // beat tick (== the tension_pull event tick)
    public int pulse;         // beat index inside the fight
    public int direction;     // -1 left / +1 right - alternates by pulse parity
    public float amplitude;   // yank strength - the 0.50/0.55/0.70 class ladder
}
/// &lt;summary&gt;One tug-of-war fight (E4 FightRecord port): the struggle window
/// spans the bite tick to the resolve tick - E3#2 keeps every fight's
/// window visible (floor 1.50s, taskbook 1-5s band).&lt;/summary&gt;
public sealed class FightRecord
{
    public double startT;             // bite tick
    public double endT;               // resolve tick
    public bool catchLanded;
    public int deslots;
    public int slipsInFight;           // T032: grips lost inside this fight (the perfect-fight tally input; a rescu
    public FishSpec spec;
    public readonly List&lt;TensionBeat&gt; Beats = new List&lt;TensionBeat&gt;();
    public double StruggleWindow { get { return endT - startT; } }
}
/// &lt;summary&gt;Frame-stepped cast line (G05-T005 core feel).&lt;/summary&gt;
public sealed class CastModel
{
    public const double StepSec = 0.05;   // canonical frame step - all E4 durations divide evenly
    public const double FlightSec = 0.20; // lure flight before splash-down (E4 graybox parity)
    public const int RecoveryPulses = 2;  // post-rescue comeback pulls (E4 graybox parity)
    /// &lt;summary&gt;Bite promise law: every cast bites within 3s of its splash ("three seconds a cast").&lt;/summ
    public const double BiteWithinSec = 3.0;
    public double Now;                        // logical clock, seconds
    public CastPhase Phase = CastPhase.Idle;
    public CastOutcome Outcome = CastOutcome.None;
    public int Casts, Catches, DeslotEvents;
    public int Rescues;
    public int TugsHeld, TugSlips;
    public BandChoice SelectedBand;   // T030: the lane this cast fishes (None = frozen classic schedule)
    public readonly List&lt;CastEvent&gt; Events = new List&lt;CastEvent&gt;();
```

```
public readonly List&lt;FightRecord&gt; Fights = new List&lt;FightRecord&gt;();
public FightRecord CurrentFight;                 // stays after resolve (rescue reads it, T008)
public AbsurdCodex Codex = new AbsurdCodex();   // single-source blind-box rotation (WaterModel law)
public WaterBodyData Water;                     // optional dense water: the bite pulse is real ripple state
public string LastCatchItem;                    // T033: the item id of the latest landed catch (records-book inp
// Slice b-1 festival window state is host-injected (local month/day
// of the clock - model purity: no DateTime.Now inside the model; the
// shipped host feeds the live clock, the checks feed pinned dates).
// 0/0 (never fed) = window closed = the frozen classic stream stays
// byte-identical to every existing gate.
public int EventMonth;
public int EventDay;
// Festival catch cursor: counts window classic catches only (band
// and out-of-window catches never move it - isolation law).
public int EventCursor;
// Slice b-2 stamp booking: the host feeds the local year the same way
// it feeds month/day, and owns the card instance (save/load rides the
// host save wire in a later slice). A null card or a never-fed year
// (0) books nothing - the classic stream stays byte-identical.
public int EventYear;
public StampCard FestivalCard;
// Slice b-3 dex trio: festival slot i lights when the window cursor
// lands it (the limited fish's dex entry - the classic 12-slot blind
// box never moves, isolation law). All-false default keeps the
// closed-window stream byte-identical; persistence rides the host
// save wire (limitedSeenMask).
public bool[] LimitedSeen = new bool[HalloweenEvent.LimitedSlotCount];
FishSpec spec;
bool[] tugPlan;
int pullIndex;
int slips;
double phaseEnd;
int tensionUnits;     // T029 tension table: held-beat units on a deep heavyweight fight
int recoverStreak;    // T029 slip-hook slope: consecutive good answers after the first slip
bool forgiven;        // T029 slip-hook slope: one forgiveness per fight
int bandCatches;       // T030: landed-catch cursor for the authored per-band item table
public void Log(string type, string detail) { Events.Add(new CastEvent(Now, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public double TimeOfFirstEvent()
{
    return Events.Count &gt; 0 ? Events[0].t : -1.0;
}
/// &lt;summary&gt;Starts one classic-schedule cast (frozen path, byte-parity
/// law - every existing gate reads it).&lt;/summary&gt;
public void BeginCast(bool[] tugPlan)
{
```

```
    BeginCast(tugPlan, BandChoice.None);
}
/// &lt;summary&gt;Starts one cast down a chosen lane (G05-T030). tugPlan[i]
/// = true when the player times pull #i correctly; a short plan misses
/// the remaining pulls. band None keeps the frozen classic schedule;
/// Shallow/Deep fish the authored band lane - the pick lands on the
/// same tick as the splash so the odds face is visible from beat one.&lt;/summary&gt;
public void BeginCast(bool[] tugPlan, BandChoice band)
{
    if (Phase == CastPhase.Flight || Phase == CastPhase.Sink || Phase == CastPhase.Fight) return;
    Casts++;
    SelectedBand = band;
    this.tugPlan = tugPlan ?? new bool[0];
    spec = band == BandChoice.None
        ? FishTable.SpecFor(Casts - 1)
        : BandSpecs.SpecFor(band, Casts - 1);
    slips = 0;
    pullIndex = 0;
    tensionUnits = 0;
    recoverStreak = 0;
    forgiven = false;
    Outcome = CastOutcome.None;
    CurrentFight = null;
    if (band != BandChoice.None)
        Log("band_pick", band == BandChoice.Shallow
            ? "shallow lane - light quick bites"
            : "deep lane - heavy haul, high risk");
    Log("cast_splash", "line flies out");
    Phase = CastPhase.Flight;
    phaseEnd = Now + FlightSec;
}
/// &lt;summary&gt;Advances the line by dt seconds. Feed the canonical 0.05
/// step for exact graybox parity. The clock snaps to the phase boundary
/// on fire: accumulated step error can never shift an event (E4 times
/// stay exact - bite 0.85/1.05/1.25, resolve 2.35/3.25/4.75).&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt;= 0.0) return;
    if (Phase != CastPhase.Flight &amp;&amp; Phase != CastPhase.Sink &amp;&amp; Phase != CastPhase.Fight) return
    Now += dt;
    while (Now &gt;= phaseEnd &amp;&amp; Phase != CastPhase.Resolved)
    {
        Now = phaseEnd;
        if (Phase == CastPhase.Flight)
        {
            Log("lure_land", "plop");
            Phase = CastPhase.Sink;
            phaseEnd += spec.sinkDur;
        }
        else if (Phase == CastPhase.Sink)
```

```
            {
                OnBite();
            }
            else if (Phase == CastPhase.Fight)
            {
                OnPulseEnd();
            }
        }
    }
    void OnBite()
    {
        // feedback triple on the bite tick (red line 11): ripple + sfx + alert, delta = 0
        Log("bite", "something pulls");
        Log("ripple", "water ring spreads");
        Log("sfx_pluck", "boing");
        Log("alert_pop", "!");
        if (Water != null) Water.PulseRipple();
        CurrentFight = new FightRecord { startT = Now, spec = spec };
        Phase = CastPhase.Fight;
        phaseEnd = Now + spec.pulseDur;
    }
    void OnPulseEnd()
    {
        Log("tension_pull", "fish yanks #" + pullIndex);
        // tension beat (T006): the yank the rod view renders - deterministic
        // L/R alternation, weight-class amplitude (zero RNG law holds)
        if (CurrentFight != null)
        {
            CurrentFight.Beats.Add(new TensionBeat
            {
                t = Now,
                pulse = pullIndex,
                direction = (pullIndex &amp; 1) == 0 ? 1 : -1,
                amplitude = (float)spec.pulseDur,
            });
        }
        bool sprintBeat = spec.deepFight &amp;&amp; spec.sprintPulse &gt;= 0
            &amp;&amp; (pullIndex == spec.sprintPulse || (spec.sprintPulse2 &gt;= 0 &amp;&amp; pullIndex == spec.spr
        bool held = pullIndex &lt; tugPlan.Length &amp;&amp; tugPlan[pullIndex];
        if (sprintBeat)
        {
            // T029 escape-sprint beat: the fish runs - holding the rod maxes
            // the tension and blows the grip, paying out line is the answer.
            if (held)
            {
                // F1 grip law: in a double-sprint fight the blowout rips the
                // rod twice - holding either run costs two grips, so a
                // never-release player provably loses the late heavyweight.
                int blowoutGrips = spec.sprintPulse2 &gt;= 0 ? 2 : 1;
                slips += blowoutGrips;
```

```
            TugSlips += blowoutGrips;
            Log("line_slip", "sprint blowout - held the rod while the fish ran");
            Log("tension_blow", "fish sprints, tension maxes, one grip gone");
            recoverStreak = 0;
        }
        else
        {
            Log("line_paid", "line paid out - the sprint runs out of road");
            if (spec.tensionCap &gt; 0 &amp;&amp; tensionUnits &gt; 0) tensionUnits--;
            recoverStreak++;
        }
    }
    else if (held)
    {
        TugsHeld++;
        Log("tug_hold", "held the line");
        if (spec.deepFight &amp;&amp; spec.tensionCap &gt; 0)
        {
            // T029 tension table: every held beat loads the rod - a full
            // table costs one beat even for a perfect player. Deep-fight
            // gate keeps classic fights byte-identical to the frozen E4
            // cycle (rounds 0-5) - the table only wakes in deep water.
            tensionUnits++;
            if (tensionUnits &gt; spec.tensionCap)
            {
                slips++;
                TugSlips++;
                Log("line_slip", "tension table full - one beat lost");
                Log("tension_full", "the rod pays for the held beat");
                recoverStreak = 0;
            }
            else recoverStreak++;
        }
        else recoverStreak++;
    }
    else
    {
        slips++;
        TugSlips++;
        Log("line_slip", "grip lost");
        recoverStreak = 0;
    }
    // T029 slip-hook slope: after the first slip, three consecutive good
    // answers (a held tug or a paid-out sprint) forgive one slip - the
    // hook slides instead of cliffing, once per fight.
    if (slips &gt; 0 &amp;&amp; !forgiven &amp;&amp; recoverStreak &gt;= 3)
    {
        slips--;
        forgiven = true;
        Log("grip_recover", "three good answers - the hook re-seats, one slip forgiven");
```

```
        }
        // T029 telegraph: the flash lands one beat before the sprint so the
        // escape is readable (SignalFx renders the flash on this tick).
        if (spec.deepFight &amp;amp;&amp;amp; spec.sprintPulse &gt;= 0
            &amp;amp;&amp;amp; (pullIndex + 1 == spec.sprintPulse || (spec.sprintPulse2 &gt;= 0 &amp;amp;&amp;amp; pullIndex + 1 ==
            Log("sprint_flash", "the fish flashes - it is about to run");
        pullIndex++;
        if (pullIndex &gt;= spec.pulses)
        {
            ResolveFight();
            return;
        }
        phaseEnd += spec.pulseDur;
    }
    void ResolveFight()
    {
        Phase = CastPhase.Resolved;
        if (CurrentFight != null) CurrentFight.endT = Now;
        if (CurrentFight != null) CurrentFight.slipsInFight = slips;
        if (slips &gt;= spec.slipLimit)
        {
            DeslotEvents++;
            Outcome = CastOutcome.Deslot;
            Log("deslot", "the catch slips off the hook");
            Log("rescue_offer", "One More Cast? (rewarded-ad fail-retry slot)");
            if (CurrentFight != null) CurrentFight.deslots++;
        }
        else
        {
            LandCatch();
        }
        // the fight record closes into the ledger; CurrentFight stays
        // referenced so the rescue path (T008) and the rod decay tail read it
        if (CurrentFight != null) Fights.Add(CurrentFight);
    }
    void LandCatch()
    {
        Catches++;
        Outcome = CastOutcome.Caught;
        if (CurrentFight != null) CurrentFight.catchLanded = true;
        // T030 lane catch: the authored per-band table hands out the item
        // (zero RNG); the classic rotation cursor never moves on a band
        // catch, so the classic stream stays byte-identical (isolation).
        string item;
        bool festivalCatch = false;
        int festivalSlot = 0;
        if (SelectedBand == BandChoice.None)
        {
            if (HalloweenEvent.IsOpen(EventMonth, EventDay))
            {
```

```
            // Slice b-1 festival routing: inside the host-fed window a
            // classic catch rides the deterministic 4-of-5 limited
            // schedule (cursor counts window classic catches only);
            // the fifth window cast keeps the classic blind box so the
            // base catalog still shows up during the festival.
            int cursor = EventCursor;
            EventCursor++;
            if (HalloweenEvent.CatchIsLimited(cursor))
            {
                festivalCatch = true;
                festivalSlot = HalloweenEvent.LimitedIndexFor(cursor);
                item = HalloweenEvent.LimitedItems[festivalSlot];
            }
            else item = Codex.RecordCatch();
        }
        else item = Codex.RecordCatch();
    }
    else item = Codex.RecordCatchAt(BandSpecs.ItemIndexFor(SelectedBand, bandCatches++));
    LastCatchItem = item;   // T033: the records book reads the landed item off the model (no Log face)
    // delight triple on the same tick + blind-box catalog entry + share-card moment
    Log("catch_pop", "hauled up: " + item);
    Log("sfx_splash", "sploosh");
    Log("count_up", "catches=" + Catches);
    if (festivalCatch)
    {
        Log("limited_catch", item + " festival slot " + (festivalSlot + 1) + "/" + HalloweenEvent.LimitedSlotCou
        LimitedSeen[festivalSlot] = true;   // b-3 dex trio: the entry books on the same tick
        // Slice b-2: the day's first festival catch books one stamp on
        // the host-owned card - same-tick face event, reward callout at
        // the goal (later catches the same day just fish, zero RNG).
        if (FestivalCard != null &amp;amp;&amp;amp; FestivalCard.RecordLimitedCatch(EventYear, EventMonth, EventDay))
            Log("stamp_book", "festival stamp " + FestivalCard.Stamps + "/" + HalloweenEvent.StampGoal
                + (FestivalCard.RewardDue ? " - reward due" : ""));
    }
    else
        Log("catalog_entry", item + " entry " + Codex.DistinctCount() + "/" + AbsurdCodex.Items.Length);
    Log("photo_card", "share card: look what I reeled up - " + item);
}
/// &lt;summary&gt;Rescue (T008, E4 parity): the rewarded-ad fail-retry slot
/// revives the just-deslotted fight with two comeback pulls, then the
/// catch lands. Valid only right after a deslot resolve (the CTA holds
/// through the fail clip); the ad category gate lives in the rescue
/// gate host - the model only books the outcome. The comeback pulls are
/// logged beats, not scored tugs: E4 Stage2 accounting counts rescues,
/// never re-tallies the tug stats (bare-play streams stay byte-identical).&lt;/summary&gt;
public bool AcceptRescue(bool[] recoveryPlan)
{
    if (Phase != CastPhase.Resolved || Outcome != CastOutcome.Deslot) return false;
    if (CurrentFight == null || CurrentFight.deslots == 0 || CurrentFight.catchLanded) return false;
    Rescues++;
```

```
            Log("rescue_accept", "One More Cast accepted");
            Log("ad_break", "rewarded video");
            Log("line_recovery", "hook re-seats");
            for (int i = 0; i &lt; RecoveryPulses; i++)
            {
                Now += CurrentFight.spec.pulseDur;
                Log("tension_pull", "comeback yank #" + i);
                if (recoveryPlan != null &amp;&amp; i &lt; recoveryPlan.Length &amp;&amp; recoveryPlan[i]) Log("tug_hold
                else Log("line_slip", "grip lost");
            }
            CurrentFight.endT = Now;
            LandCatch();
            return true;
        }
    }
}
// CatchClipView.cs
// CatchClipView.cs - procedural placeholder for the resolve-moment clips (G05-T007).
// The comedy payoff of every cast: a landed catch pops the absurd item - the
// blind-box first laugh (the rice cooker opens the show) - with a delight ring
// and a photo-card beat; a deslot plops the catch back with a fail splash pair
// and the rescue CTA. Both clips fire on the resolve tick itself (same-frame
// feedback, red line 11) and read the model's logical clock, never wall time,
// so headless gates see the exact clip the player sees. X027 wiring rule:
// "catch_&lt;item&gt;" art stems upgrade in place at S2.5; procedural fallbacks are
// null-safe so gates never break on missing art.
using System;
using UnityEngine;
using Sy.Core;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;Builds and drives the catch-outcome clips. Call Rebuild once,
    /// then Apply(model) after every model step (ApplyAt feeds an explicit clock
    /// for hosts that blend clocks; gates probe clip ages through it).&lt;/summary&gt;
    public sealed class CatchClipView
    {
        public const double PopUpSec = 0.15;              // pops in from PopInFloor - visible on tick 0
        public const double HoldSec = 0.45;               // the laugh beat holds
        public const double EaseOffSec = 0.40;            // then eases away
        public const double ClipTotalSec = PopUpSec + HoldSec + EaseOffSec;   // 1.00s clip window
        public const float PopInFloor = 0.30f;            // tick-0 scale floor - same-frame visibility law
        public const float ItemBaseScale = 2.2f;
        public const float ItemY = 17.5f;                 // pops between rod butt and tip
        public const float CardX = 3.1f;
        public const float SplashY = 14.5f;               // on the water line where the hook slips
        public const float CtaY = 12.0f;
        static readonly Color RingColor = Hex(0xE8833A);   // buoy warm orange - the pitch accent
        static readonly Color SplashColor = Hex(0x9FB8C4); // grey-teal plop rings
        static readonly Color CtaColor = Hex(0xE8833A);    // rescue CTA in the accent
        static readonly Color CtaCappedColor = Hex(0x6B6B6B); // budget spent - the honest grey
```

```
static readonly Color CardColor = Hex(0xF5EFE0);   // polaroid cream
readonly Transform root;
static Sprite solid;
static Sprite circle;
public CatchClipView(Transform root) { this.root = root; }
/// &lt;summary&gt;Builds the clip rig: the delight ring, the hauled item, the
/// photo-card beat, the fail splash pair and the rescue CTA. Returns the
/// number of objects created.&lt;/summary&gt;
public int Rebuild()
{
    Clear();
    int built = 0;
    built += Make(
        Circle,
        RingColor,
        "clip_ring",
        new Vector3(0f, ItemY, 0f),
        new Vector3(1.6f, 1.6f, 1f),
        8) != null ? 1 : 0;
    Sprite itemArt = ArtRuntime.Get("catch_" + AbsurdCodex.Items[0]);
    Sprite itemFace = itemArt != null ? itemArt : Circle;
    built += Make(
        itemFace,
        Color.white,
        "clip_item",
        new Vector3(0f, ItemY, 0f),
        WaterView.FitScale(itemFace, ItemBaseScale, ItemBaseScale),
        9) != null ? 1 : 0;
    built += Make(
        Solid,
        CardColor,
        "clip_card",
        new Vector3(CardX, ItemY, 0f),
        new Vector3(1.1f, 1.4f, 1f),
        9) != null ? 1 : 0;
    for (int k = 0; k &lt; 2; k++)
    {
        built += Make(
            Circle,
            SplashColor,
            "fail_splash_" + k,
            new Vector3(0f, SplashY, 0f),
            new Vector3(1f, 1f, 1f),
            8) != null ? 1 : 0;
    }
    built += Make(
        Solid,
        CtaColor,
        "rescue_cta",
        new Vector3(0f, CtaY, 0f),
```

```
        new Vector3(2.6f, 0.7f, 1f),
        10) != null ? 1 : 0;
    return built;
}
/// &lt;summary&gt;Per-step update on the model clock: drives the catch clip or
/// the fail clip from the latest resolve event. No live resolve leaves
/// the whole rig dark.&lt;/summary&gt;
public void Apply(CastModel m)
{
    ApplyAt(m, m == null ? 0.0 : m.Now);
}
/// &lt;summary&gt;Per-step update on an explicit clock (hosts may blend; the
/// gate probes clip ages the same way). age &amp;lt; 0 or past the window
/// darkens the rig.&lt;/summary&gt;
public void ApplyAt(CastModel m, double now)
{
    if (root == null || m == null) return;
    var item = FindChild("clip_item");
    if (item == null) return;
    var resolve = LatestResolve(m);
    double age = resolve == null ? -1.0 : now - resolve.t;
    bool live = ClipLive(age);
    bool catchClip = live &amp;&amp; resolve.type == "catch_pop";
    bool failClip = live &amp;&amp; resolve.type == "deslot";
    // catch clip: the hauled absurd item pops in the slot color with the
    // delight ring; the photo-card beat snaps in once the pop lands
    var sr = item.GetComponent&lt;SpriteRenderer&gt;();
    if (catchClip)
    {
        // Festival face (c-2 tail, the B994 slot-alias family): a limited
        // landing never enters the classic codex rotation, so the slot
        // lookup below would alias it to the stale classic slot (and a
        // first-cast festival landing with an unmoved cursor would index
        // Items[-1]) - when the festival face is live read the landed id
        // straight off the model; the classic path keeps the
        // byte-identical codex-slot face (zero drift law).
        bool festival = HalloweenEvent.IsOpen(m.EventMonth, m.EventDay)
            &amp;&amp; HalloweenEvent.IsLimitedItem(m.LastCatchItem);
        int idx = LastCatchIndex(m);
        string clipItem = festival
            ? m.LastCatchItem
            : AbsurdCodex.Items[idx];
        Sprite art = ArtRuntime.Get("catch_" + clipItem);
        sr.sprite = art != null ? art : Circle;
        sr.color = festival
            ? (art != null ? Color.white : Hex(WaterView.PumpkinBobberHex))
            : WaterView.ItemColor(idx);
        float s = ItemScaleFor(age);
        item.transform.localScale = WaterView.FitScale(sr.sprite, s, s);
    }
```

```
        item.SetActive(catchClip);
        var ring = FindChild("clip_ring");
        if (ring != null)
        {
            float rs = RingScaleFor(age);
            ring.transform.localScale = new Vector3(rs, rs, 1f);
            ring.SetActive(catchClip);
        }
        var card = FindChild("clip_card");
        if (card != null) card.SetActive(catchClip &amp;&amp; age &gt;= PopUpSec);
        // fail clip: the catch plops back - the splash pair spreads on the
        // water line and the rescue CTA holds through the clip (the
        // fail-retry slot; T008 wires the tap into the category gate)
        for (int k = 0; k &lt; 2; k++)
        {
            var splash = FindChild("fail_splash_" + k);
            if (splash != null)
            {
                float s = SplashScaleFor(age, k);
                splash.transform.localScale = new Vector3(s, s, 1f);
                splash.SetActive(failClip);
            }
        }
        var cta = FindChild("rescue_cta");
        if (cta != null) cta.SetActive(failClip);
    }
    /// &lt;summary&gt;Budget face for the rescue CTA (T008): the accent orange
    /// while the rescue category still has daily budget, the honest grey
    /// once capped - the tap gate refuses anyway, the color never lies.
    /// No-op on a missing CTA (null-safe gate law).&lt;/summary&gt;
    public void ApplyCtaBudget(GameObject cta, bool budgetOk)
    {
        if (cta == null) return;
        var sr = cta.GetComponent&lt;SpriteRenderer&gt;();
        if (sr != null) sr.color = budgetOk ? CtaColor : CtaCappedColor;
    }
    /// &lt;summary&gt;Latest resolve event (catch_pop or deslot), null when none.&lt;/summary&gt;
    public static CastEvent LatestResolve(CastModel m)
    {
        if (m == null) return null;
        for (int i = m.Events.Count - 1; i &gt;= 0; i--)
        {
            string t = m.Events[i].type;
            if (t == "catch_pop" || t == "deslot") return m.Events[i];
        }
        return null;
    }
    /// &lt;summary&gt;Codex slot of the latest landed catch (Order counts catches;
    /// -1 before the first landing).&lt;/summary&gt;
    public static int LastCatchIndex(CastModel m)
```

```
{
    if (m == null || m.Codex.Order &lt;= 0) return -1;
    return (m.Codex.Order - 1) % AbsurdCodex.Items.Length;
}
/// &lt;summary&gt;Clip liveness law: live while age sits inside [0, ClipTotalSec].&lt;/summary&gt;
public static bool ClipLive(double age)
{
    return age &gt;= 0.0 &amp;&amp; age &lt;= ClipTotalSec;
}
/// &lt;summary&gt;Item scale envelope by age: pops in from PopInFloor (visible
/// on tick 0), holds full, eases to zero by the window end.&lt;/summary&gt;
public static double ClipEnvelope(double age)
{
    if (age &lt; 0.0) return 0.0;
    if (age &lt;= PopUpSec) return PopInFloor + (1.0 - PopInFloor) * (age / PopUpSec);
    if (age &lt;= PopUpSec + HoldSec) return 1.0;
    if (age &lt;= ClipTotalSec) return 1.0 - (age - PopUpSec - HoldSec) / EaseOffSec;
    return 0.0;
}
/// &lt;summary&gt;Hauled-item scale for a resolve at the given age - the exact
/// number the gate re-derives (single-source law, no formula duplication).&lt;/summary&gt;
public static float ItemScaleFor(double age)
{
    return ItemBaseScale * (float)ClipEnvelope(age);
}
/// &lt;summary&gt;Delight ring grows through the clip window (starts visible at
/// tick 0 - same-frame law).&lt;/summary&gt;
public static float RingScaleFor(double age)
{
    double p = Math.Min(Math.Max(age, 0.0) / ClipTotalSec, 1.0);
    return 1.6f + 1.6f * (float)p;
}
/// &lt;summary&gt;Fail splash scale by age: inner ring k=0, outer ring k=1 -
/// the plop spreads as the clip plays.&lt;/summary&gt;
public static float SplashScaleFor(double age, int ring)
{
    double p = Math.Min(Math.Max(age, 0.0) / ClipTotalSec, 1.0);
    return ring == 0 ? 0.8f + 1.4f * (float)p : 1.6f + 2.8f * (float)p;
}
/// &lt;summary&gt;Looks up a built child object by name. Null when absent.&lt;/summary&gt;
public GameObject FindChild(string childName)
{
    if (root == null) return null;
    Transform t = root.Find(childName);
    return t == null ? null : t.gameObject;
}
public void Clear()
{
    if (root == null) return;
    for (int i = root.childCount - 1; i &gt;= 0; i--) WaterView.SafeDestroy(root.GetChild(i).gameObject);
```

```
        }
        static Sprite Solid
        {
            get { if (solid == null) solid = MakeSprite(4, false); return solid; }
        }
        static Sprite Circle
        {
            get { if (circle == null) circle = MakeSprite(32, true); return circle; }
        }
        static Sprite MakeSprite(int size, bool round)
        {
            var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
            float r = size * 0.5f;
            for (int y = 0; y &lt; size; y++)
            {
                for (int x = 0; x &lt; size; x++)
                {
                    float dx = x + 0.5f - r;
                    float dy = y + 0.5f - r;
                    bool inside = !round || dx * dx + dy * dy &lt;= r * r;
                    tex.SetPixel(x, y, inside ? Color.white : Color.clear);
                }
            }
            tex.Apply();
            return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
        }
        GameObject Make(Sprite sprite, Color color, string name, Vector3 localPos, Vector3 scale, int sortingOrder)
        {
            if (root == null) return null;
            var go = new GameObject(name);
            go.transform.SetParent(root, false);
            go.transform.localPosition = localPos;
            go.transform.localScale = scale;
            var sr = go.AddComponent&lt;SpriteRenderer&gt;();
            sr.sprite = sprite;
            sr.color = color;
            sr.sortingOrder = sortingOrder;
            return go;
        }
        static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
    }
}
// DailyCast.cs
// DailyCast.cs - daily one-cast challenge for ReelRiot (G05-T017).
// Every day the pond hands every player the same special rod: the rod's
// rhythm derives from the very day seed the ad cap, the save and the pond
// rotation already flip on (T011 one-boundary law), so the whole world faces
```

```
// the identical tug plan, the identical fish and the identical settled
// outcome - a system-preset daily rotation in pure offline form (X032: no
// server ever ships the question of the day). One cast settles the day: the
// rod plays its own rhythm with zero player skill, and the result folds
// into the very replay-code pipeline the rivalry channel already ships
// (T012) - the result card embeds a code any device decodes and re-simulates
// byte-identically (local recompute, zero upload). The rod's privilege is
// its built-in comeback: a day whose stream deslots still lands the catch
// when the player takes the one rescue - the comeback pulls are the rod's
// own, perfect by construction. Daily-used accounting rides the local save
// wire (T011): one marker plus the day's catch index, freed by the next day
// key through string compare alone, surviving restarts.
using System;
using Sy.Platform;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;The day's privilege rod: its rhythm is the day's tug plan
    /// and its comeback is perfect. Pure data, no engine types.&lt;/summary&gt;
    public sealed class DailyRod
    {
        public int index;          // 0..6 - the day's rod slot in the cycle
        public string label = "";  // short English rod name (telemetry face)
        public int seed;           // the day seed (shared T011 boundary)
        public bool[] plan;        // the day's rhythm: PlanFor(seed, 0)
    }
    /// &lt;summary&gt;One settled daily cast: the outcome faces plus the share code
    /// that reproduces the whole stream on any device.&lt;/summary&gt;
    public sealed class DailyResult
    {
        public DailyRod rod;
        public bool caught;
        public int catchIndex = -1;  // catalog slot, -1 = the day slipped it away
        public string item = "";     // catalog id, "" on deslot
        public string code = "";     // replay code embedding seed/casts/rescues
        public CastModel model;      // the settled one-cast stream (event ledger)
    }
    public static class DailyCast
    {
        public const int CastsPerDay = 1;   // one cast decides the day
        public const int RodCycle = 7;
        public const int MaxRescues = 1;    // the rod carries one built-in comeback
        // Rod names ride the RodTexts active face (T042-a): the English
        // table stays byte-identical to the old embedded array so every
        // gate signature holds; zh rides the data file once the font
        // re-cast slice covers the new glyphs (gambling-token hygiene
        // stays checker-side).
        /// &lt;summary&gt;The day's rod: label slot and tug plan both derive from the
        /// shared day seed - the same key hands every player the same rod
        /// (malformed keys degrade to seed 0, never throw).&lt;/summary&gt;
        public static DailyRod RodFor(string dayKey)
```

```
{
    int seed = PondRotation.DaySeed(dayKey);
    int i = seed % RodCycle;
    if (i &lt; 0) i += RodCycle;
    return new DailyRod
    {
        index = i,
        label = RodTexts.RodName(i),
        seed = seed,
        plan = ReplayCodec.PlanFor(seed, 0),
    };
}
/// &lt;summary&gt;Today's rod (device-local clock - zero server faces).&lt;/summary&gt;
public static DailyRod TodayRod() =&gt; RodFor(AdCapPolicy.TodayKey());
/// &lt;summary&gt;Settles the day's one cast: a fresh one-cast replay of the
/// day seed - the rod plays its own rhythm and the optional rescue is
/// the rod's perfect comeback. Deterministic on any device.&lt;/summary&gt;
public static DailyResult Play(string dayKey, int rescues)
{
    DailyRod rod = RodFor(dayKey);
    var run = new ReplayRun { seed = rod.seed, casts = CastsPerDay, rescues = rescues &gt; 0 ? MaxRescues : 0 };
    CastModel m = ReplayCodec.RunReplay(run);
    int idx = CatchClipView.LastCatchIndex(m);
    return new DailyResult
    {
        rod = rod,
        caught = m.Outcome == CastOutcome.Caught,
        catchIndex = idx,
        item = idx &gt;= 0 &amp;&amp; idx &lt; AbsurdCodex.Items.Length ? AbsurdCodex.Items[idx] : "",
        code = ReplayCodec.Encode(run),
        model = m,
    };
}
/// &lt;summary&gt;Canonical rod signature - the dual-derive byte-parity
/// instrument (two derives of one day agree here or nowhere).&lt;/summary&gt;
public static string Signature(DailyRod r)
{
    if (r == null) return "null";
    var sb = new System.Text.StringBuilder();
    sb.Append("r").Append(r.index).Append('|').Append(r.label).Append("|s:").Append(r.seed).Append("|p:");
    if (r.plan == null) { sb.Append("null"); return sb.ToString(); }
    for (int i = 0; i &lt; r.plan.Length; i++) sb.Append(r.plan[i] ? '1' : '0');
    return sb.ToString();
}
// ---- daily-once accounting (rides the T011 save wire) ----
/// &lt;summary&gt;The daily cast is spent when the save's marker equals the
/// shared day key (string compare - the next local midnight frees the
/// cast by itself, no reset beat needed).&lt;/summary&gt;
public static bool UsedToday() =&gt; SaveWire.State.dailyCastKey == AdCapPolicy.TodayKey();
/// &lt;summary&gt;The catch today's settled cast landed (-1 = slipped or none).&lt;/summary&gt;
```

```
        public static int TodayCatch() =&gt; UsedToday() ? SaveWire.State.dailyCastCatch : -1;
        /// &lt;summary&gt;Books the settled daily cast: one marker plus the day's
        /// catch index, persisted on the booking beat itself.&lt;/summary&gt;
        public static void MarkUsed(DailyResult r)
        {
            SaveWire.OnDailyCast(r == null ? -1 : r.catchIndex);
        }
    }
}
// DexExpandGate.cs
// DexExpandGate.cs - the ad-gated dex expansion (G05-T015).
// The completion beat pays forward: the T014 one-slot-short itch primes the
// player to finish the album, and the very resolve that completes the 12/12
// codex books the expand offer on that tick - the dex can hold twice the
// absurd (12 -&gt; 24 slots, the LiveOps rotating-category landing row). The
// chain is the RescueGate law verbatim: HasCategoryBudget precheck on the
// same policy bucket the registered service consumes, category-capped
// ShowRewarded on the g05_dex_expand 3/day category, the evt_ad funnel face
// (offer -&gt; show -&gt; complete -&gt; dex_expanded) for the S2 telemetry promotion.
// The armed guard is state law, not a beat: a complete album that is not
// expanded yet stays tappable (a returning player with a loaded 12/12 codex
// never misses the face), and once expanded the point goes dark forever -
// the unlock is permanent, the save books it on the beat itself. The gate
// never touches the cast model: the event stream stays byte-identical
// whether the offer books or not (the check proves it bare-play). Zero RNG.
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;The dex-expand gate: books the offer on the album-completion
    /// resolve tick, gates the expand CTA tap through the ad category chain,
    /// unlocks the 24-slot page when the ad completes.&lt;/summary&gt;
    public sealed class DexExpandGate
    {
        /// &lt;summary&gt;Expand point id (T015): draws from its own 3/day
        /// g05_dex_expand category - the unlock is permanent, the cap is a
        /// farming safety valve, never a re-sell.&lt;/summary&gt;
        public const string ExpandPoint = "g05_dex_expand";
        readonly CastModel model;
        readonly AdCapPolicy adCap;
        readonly IAdsService ads;
        readonly Func&lt;string&gt; dayKey;
        double lastOfferT = double.NegativeInfinity;   // one offer beat per resolve tick
        int lastDex = -1;                              // dex count observed at the previous booked resolve
        public readonly List&lt;string&gt; AdFunnel = new List&lt;string&gt;();
        /// &lt;summary&gt;In-memory unlock mirror; the save is the authority (the
        /// host seeds this from SaveWire.State.dexExpanded at wiring time -
        /// boot loads the save first, T011 law).&lt;/summary&gt;
```

```
public bool Expanded;
public DexExpandGate(CastModel model, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = null)
{
    this.model = model;
    this.adCap = adCap;
    this.ads = ads;
    // the precheck reads the very bucket the registered service consumes
    // (injected key only for deterministic gates; production rides TodayKey)
    this.dayKey = dayKey ?? AdCapPolicy.TodayKey;
    if (ads != null) ads.OnAdEvent += OnAdLifecycle;
}
/// &lt;summary&gt;Live expand-category precheck: always-fresh budget face on
/// the same policy instance and the same day bucket the registered ads
/// service consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
public bool OfferBudgetOk
{
    get
    {
        return adCap != null
            &amp;&amp; adCap.HasCategoryBudget(dayKey(), AdCategories.CategoryOfPoint(ExpandPoint));
    }
}
/// &lt;summary&gt;True while the album is complete and not yet expanded - the
/// persistent IAA face a tap may enter the chain on. Below 12/12 stays
/// dark (the T014 hook owns the one-away beat), an expanded codex
/// never sells the same row twice.&lt;/summary&gt;
public bool ExpandArmed
{
    get
    {
        return !Expanded
            &amp;&amp; model != null
            &amp;&amp; model.Codex != null
            &amp;&amp; model.Codex.DistinctCount() == AbsurdCodex.Items.Length;
    }
}
string Stamp()
{
    return model == null ? "0.00" : model.Now.ToString("0.00");
}
/// &lt;summary&gt;Offer booking on the album-completion resolve tick (host
/// contract, same-frame law as ShareLoop.NoteResolve): the catch that
/// lifts the codex from one-slot-short to the full 12/12 books its
/// offer on this very tick - capped offers book too, the honest face
/// refuses at the tap, never after the ad. 11/12 and below stay
/// silent (the dex-hook owns that beat); a deslot never offers; an
/// expanded codex never re-offers; a codex that arrived complete from
/// the save books nothing (its face is the armed CTA, not a beat).&lt;/summary&gt;
public void NoteResolve()
{
```

```
            var resolve = CatchClipView.LatestResolve(model);
            if (model == null || resolve == null || resolve.type != "catch_pop") { if (model != null) lastDex = DexOf(mo
            int dex = DexOf(model);
            bool completion = dex == AbsurdCodex.Items.Length &amp;amp;&amp;amp; lastDex == AbsurdCodex.Items.Length - 1;
            lastDex = dex;
            if (!completion || Expanded || resolve.t == lastOfferT) return;
            lastOfferT = resolve.t;
            bool ok = OfferBudgetOk;
            AdFunnel.Add("evt_ad_offer" + (ok ? "" : "_capped") + "@" + Stamp());
        }
        static int DexOf(CastModel m)
        {
            return m == null || m.Codex == null ? -1 : m.Codex.DistinctCount();
        }
        /// &lt;summary&gt;CTA tap: armed guard - budget precheck - rewarded view -
        /// permanent unlock. The ad only ever plays on an armed complete album
        /// with budget left; a capped tap refuses BEFORE the ad ever plays
        /// (offer_capped, no show - the honest-face law); the unlock books
        /// the save on the beat itself.&lt;/summary&gt;
        public async Task&lt;bool&gt; AcceptFromCta()
        {
            if (ads == null) return false;
            if (!ExpandArmed) return false;
            if (!OfferBudgetOk)
            {
                AdFunnel.Add("evt_ad_offer_capped@" + Stamp());
                return false;
            }
            RewardResult r = await ads.ShowRewarded(ExpandPoint);
            if (r == null || !r.Completed) return false;
            Expanded = true;
            SaveWire.OnDexExpanded();
            AdFunnel.Add("evt_dex_expanded@" + Stamp());
            return true;
        }
        /// &lt;summary&gt;evt_ad funnel (T015): lifecycle events of the expand point
        /// land in the funnel list for gate assertions; S2 promotes this face
        /// to the unified telemetry pipeline.&lt;/summary&gt;
        void OnAdLifecycle(AdLifecycle e)
        {
            if (e.PointId != ExpandPoint) return;
            AdFunnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + Stamp());
        }
    }
}
// DexHookView.cs
// DexHookView.cs - the one-slot-short dex hook face (G05-T014).
// The core-loop itch (taskbook S2-T014): when the absurd codex sits at
// exactly 11/12 - one ridiculous item short of the full album - every
// landed catch resolve arms the hook on that very tick: a spotlight ring
```

```
// locks onto the missing dex slot and the "ONE MORE!" bait line rides
// right above it, both visible on tick 0 (same-frame law, red line 11),
// settling through a deterministic window. The full album (12/12) never
// triggers - completion is its own moment - and anything short of
// one-away (&lt;=10/12) never triggers: the itch only exists when this next
// cast could finish the set, which is what keeps the one-away player
// casting (the T015 dex-expand conversion inherits this exact moment). A
// deslot never arms - the fail clip owns that beat with its rescue CTA,
// one message per beat. Arming law mirrors ShareLoop.NoteResolve: the
// host books every resolve through NoteResolve(model) on the resolve
// tick; the model event stream stays read-only (bare-play byte parity
// holds whether the hook fires or not). Zero RNG faces anywhere.
using System;
using UnityEngine;
namespace Sy.ReelRiot
{
    public sealed class DexHookView
    {
        public const double SpotSec = 1.60;       // hook window: rides the 1.00s catch clip, settles after
        public const double ArmBudgetSec = 1.00;   // taskbook budget: the face must start within 1.0s of the resolve ti
        public static string BaitLine =&gt; RodTexts.Active(RodTexts.KeyDexHookBait);
        public const float SpotBase = 1.10f;      // spotlight floor - visible on tick 0
        public const float SpotSpan = 0.60f;      // breathing growth span
        public const float BaitAlphaFloor = 0.55f;
        public const float BaitAlphaSpan = 0.45f;
        public const float BaitLiftY = 1.40f;      // the bait line rides this far above the locked slot
        static readonly Color SpotColor = Hex(0xE8833A);   // buoy warm orange - the pitch accent
        static readonly Color BaitColor = Hex(0xE8833A);   // the bait line shouts in the accent
        readonly Transform root;
        static Sprite circle;
        static Font baitFont;
        public double ArmedAt = -1.0;              // resolve-tick stamp of the latest arm (-1 = never)
        public int MissingSlot = -1;              // the single unseen slot the spotlight locks
        public int ArmCount;                      // arms so far (the persistent itch ledger)
        public int LastArmDex = -1;               // dex count at the latest arm (one-short by law)
        double lastArmedResolveT = double.NegativeInfinity;   // one arm per resolve tick
        public DexHookView(Transform root) { this.root = root; }
        /// &lt;summary&gt;Resolve-tick arming (host contract, same-frame law as
        /// ShareLoop.NoteResolve): a landed catch with the codex at exactly
        /// one-slot-short (11/12) arms the hook on this very tick - the state
        /// law holds for duplicates too (a one-away returning player gets the
        /// itch on every catch until the set completes). 12/12 and &lt;=10/12
        /// stay dark; a deslot never arms (the fail clip owns that beat). A
        /// null or resolve-less model is a no-op, never a throw.&lt;/summary&gt;
        public void NoteResolve(CastModel m)
        {
            var resolve = CatchClipView.LatestResolve(m);
            if (m == null || resolve == null || resolve.type != "catch_pop") return;
            if (resolve.t == lastArmedResolveT) return;
            int dex = m.Codex.DistinctCount();
```

```
    if (dex != AbsurdCodex.Items.Length - 1) return;
    ArmedAt = resolve.t;
    MissingSlot = MissingIndexOf(m.Codex);
    LastArmDex = dex;
    ArmCount++;
    lastArmedResolveT = resolve.t;
}
/// &lt;summary&gt;The single unseen slot index, -1 unless exactly one hole.&lt;/summary&gt;
public static int MissingIndexOf(AbsurdCodex codex)
{
    if (codex == null) return -1;
    int missing = -1;
    for (int i = 0; i &lt; AbsurdCodex.Items.Length; i++)
    {
        if (codex.Seen[i]) continue;
        if (missing &gt;= 0) return -1;
        missing = i;
    }
    return missing;
}
/// &lt;summary&gt;Builds the hook rig: the missing-slot spotlight ring plus
/// the ONE MORE! bait line. Returns the number of objects created.&lt;/summary&gt;
public int Rebuild()
{
    Clear();
    int built = 0;
    built += Make(
        Circle,
        SpotColor,
        "dex_spot",
        Vector3.zero,
        Vector3.zero,
        6) != null ? 1 : 0;
    built += MakeText(
        "dex_bait",
        BaitLine,
        Vector3.zero) != null ? 1 : 0;
    return built;
}
/// &lt;summary&gt;Per-step face on the model clock.&lt;/summary&gt;
public void Apply(CastModel m)
{
    ApplyAt(m == null ? 0.0 : m.Now);
}
/// &lt;summary&gt;Per-step face on an explicit clock (hosts may blend; the
/// gate probes the same way): the spotlight snaps onto the locked slot
/// and the bait line rides above it, both visible from tick 0, dark
/// outside the window. Never-armed or unhittable states stay dark.&lt;/summary&gt;
public void ApplyAt(double now)
{
```

```
            ArmedRun = null;
            ShareFunnel.Clear();
        }
    }
}
// SignalFx.cs
// SignalFx.cs - the three hook-signal faces for ReelRiot (G05-T025, G04-T021
// all-three precedent isomorphic port). Red line 11 demands one measurable
// 10-second signal from the laugh/wow/flex family; this unit ships all three:
//  - LaughLedger (first laugh): every catch resolve books the blind-box gag -
//    the first absurd catch stamps FirstLaughAt on the very resolve tick, the
//    subtitle escalates to the "AGAIN?! SERIOUSLY?!" punchline on a streak of
//    consecutive catches, a deslot resets the streak (progress-beat reset,
//    *wipes brow* isomorph), and a streak ring grows with the streak (capped -
//    no unbounded presentation state).
//  - CloseCallLaw (first wow): the so-close trigger - a deslot while the dex
//    itch sits armed at one-slot-short ("one cast from the full album and it
//    flew off the hook"). Pure law, zero side effects: the host/gate asks, then
//    engages the ported SlowMoFx with the SO CLOSE screen tag.
//  - PerfectLaw + GoldenRodView (first flex): a run with at least one catch,
//    zero deslots and zero assists (no rescue taken, no hint revealed) flips
//    the GOLDEN ROD banner on the run's own merit and books the
//    evt_signal_perfect funnel line exactly once.
// The model event stream stays read-only everywhere (bare-play byte parity
// holds whether the faces fire or not - the NoteResolve/NoteRun contract is
// the same read-only law the dex hook already uses). Zero RNG faces.
using System;
using UnityEngine;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;First-laugh blind-box gag state + face (first hook signal).&lt;/summary&gt;
    public sealed class LaughLedger
    {
        public const double OpeningWindowSec = 10.0;   // red line 11: the first laugh lands inside the opening window
        public static string UpgradeLine =&gt; RodTexts.Active(RodTexts.KeyUpgradeLine);
        public const float RingBaseScale = 1.10f;      // streak ring floor - visible on tick 0
        public const float RingStreakStep = 0.35f;     // each consecutive absurd catch grows the ring
        public const int RingStreakCap = 4;           // growth caps - bounded presentation state
        public double FirstLaughAt = -1.0;             // resolve-tick stamp of the first absurd catch (-1 = never)
        public int Streak;                            // consecutive catch resolves (a deslot resets it)
        public int LaughBeats;                        // total catch resolves booked (the gag ledger)
        readonly Transform root;
        double lastNoteT = double.NegativeInfinity;   // one booking per resolve tick
        static readonly Color Accent = Hex(0xE8833A);  // buoy warm orange - the pitch accent
        static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
        public LaughLedger(Transform root) { this.root = root; }
        /// &lt;summary&gt;Base subtitle for a fresh absurd catch; the upgrade line
```

```
/// takes over from streak 2 (the escalating punchline law). Article law:
/// AN before a vowel-initial item, A otherwise (pure string law).&lt;/summary&gt;
public static string SubtitleFor(int streak, string itemId)
{
    if (streak &gt;= 2) return UpgradeLine;
    string name = ShareCards.ItemName(itemId).ToUpperInvariant();
    bool vowel = name.Length &gt; 0 &amp;&amp; "AEIOU".IndexOf(name[0]) &gt;= 0;
    string tmpl = RodTexts.Active(vowel ? RodTexts.KeySubtitleVowel : RodTexts.KeySubtitleConsonant);
    return tmpl.Replace("{0}", name);
}
/// &lt;summary&gt;Streak ring scale law (the gate re-derives - single source):
/// grows one step per streak beat past the first, hard-capped.&lt;/summary&gt;
public static float RingScaleFor(int streak)
{
    int capped = streak &lt;= 1 ? 0 : Math.Min(streak - 1, RingStreakCap);
    return RingBaseScale + RingStreakStep * capped;
}
/// &lt;summary&gt;Resolve-tick booking (read-only model law, same contract as
/// DexHookView.NoteResolve): a catch resolve stamps the first laugh on
/// its very tick, grows the streak and the beat ledger; a deslot resolve
/// resets the streak (the gag reads the next catch as a fresh first).
/// A null or resolve-less model is a no-op, never a throw.&lt;/summary&gt;
public void NoteResolve(CastModel m)
{
    var resolve = CatchClipView.LatestResolve(m);
    if (m == null || resolve == null) return;
    if (resolve.t == lastNoteT) return;
    lastNoteT = resolve.t;
    if (resolve.type == "catch_pop")
    {
        Streak++;
        LaughBeats++;
        if (FirstLaughAt &lt; 0.0) FirstLaughAt = resolve.t;
    }
    else if (resolve.type == "deslot")
    {
        Streak = 0;
    }
}
/// &lt;summary&gt;Builds the gag rig: the laugh subtitle + the streak ring.
/// Returns the number of objects created (frozen footprint: 2).&lt;/summary&gt;
public int Rebuild()
{
    Clear();
    int built = 0;
    built += FaceKit.MakeText(root, "laugh_text", RodTexts.Active(RodTexts.KeyLaughCore), new Vector3(0f, 20.3f,
    built += FaceKit.MakeCircle(root, "streak_ring", Accent, new Vector3(0f, 17.5f, 0f), new Vector3(RingBaseSca
    return built;
}
/// &lt;summary&gt;Per-step face on the explicit clock (the gate probes the
```

```
    /// same way): the subtitle and streak ring ride the catch clip window
    /// (single-source liveness - no second clock), dark on fail clips and
    /// after a streak-resetting deslot.&lt;/summary&gt;
    public void ApplyAt(CastModel m, double now)
    {
        var text = FaceKit.FindChild(root, "laugh_text");
        var ring = FaceKit.FindChild(root, "streak_ring");
        if (text == null || ring == null) return;
        var resolve = CatchClipView.LatestResolve(m);
        double age = resolve == null ? -1.0 : now - resolve.t;
        bool live = m != null &amp;&amp; resolve != null &amp;&amp; resolve.type == "catch_pop"
            &amp;&amp; CatchClipView.ClipLive(age) &amp;&amp; Streak &gt; 0;
        text.SetActive(live);
        ring.SetActive(live);
        if (!live) return;
        int idx = CatchClipView.LastCatchIndex(m);
        string item = idx &gt;= 0 &amp;&amp; idx &lt; AbsurdCodex.Items.Length ? AbsurdCodex.Items[idx] : "old_boot"
        var tm = text.GetComponent&lt;TextMesh&gt;();
        if (tm != null) tm.text = SubtitleFor(Streak, item);
        float s = RingScaleFor(Streak);
        ring.transform.localScale = new Vector3(s, s, 1f);
    }
    public void Clear() { FaceKit.ClearChildren(root); }
}
/// &lt;summary&gt;The so-close trigger law (first wow signal): pure state read,
/// zero side effects - the host/gate asks, then engages SlowMoFx.&lt;/summary&gt;
public static class CloseCallLaw
{
    public static string TagLine =&gt; RodTexts.Active(RodTexts.KeyCloseCallTag);
    /// &lt;summary&gt;Trigger window: a deslot resolve while the dex itch sits
    /// armed at one-slot-short - the spotlight has already locked the
    /// missing slot (ArmedAt &amp;gt;= 0, MissingSlot valid) and the codex
    /// still reads 11/12 (DexHookCheck persistent-itch state law: the arm
    /// is the resolve-tick face, the one-short state itself persists).
    /// A deslot below one-short never engages (the fail clip owns that
    /// beat alone - one message per beat).&lt;/summary&gt;
    public static bool ShouldEngage(DexHookView hook, CastModel m, double now)
    {
        var resolve = CatchClipView.LatestResolve(m);
        if (hook == null || m == null || resolve == null || resolve.type != "deslot") return false;
        if (hook.ArmedAt &lt; 0.0 || hook.MissingSlot &lt; 0) return false;
        return m.Codex.DistinctCount() == AbsurdCodex.Items.Length - 1;
    }
    /// &lt;summary&gt;Builds the lazy screen-tag TextMesh (one child, active -
    /// SlowMoFx.Restore lights it out). Returns null on a null parent.&lt;/summary&gt;
    public static TextMesh MakeTag(Transform parent)
    {
        return FaceKit.MakeText(parent, "closecall_tag", TagLine, new Vector3(0f, 16.0f, -1f), Hex(0xF5EFE0), 26, 0.
    }
    static Color Hex(int rgb)
```

```
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
    }
    /// &lt;summary&gt;The golden-rod guard (first flex signal): pure law.&lt;/summary&gt;
    public static class PerfectLaw
    {
        /// &lt;summary&gt;A perfect run: at least one landed catch, zero deslots and
        /// zero assists (no rescue taken, no hint revealed) - the flex signal
        /// flips on the run's own merit only.&lt;/summary&gt;
        public static bool IsPerfect(CastModel m, int hintsUsed)
        {
            return m != null &amp;&amp; m.Catches &gt;= 1 &amp;&amp; m.DeslotEvents == 0 &amp;&amp; m.Rescues == 0 &amp;
        }
    }
    /// &lt;summary&gt;The golden banner + CTA face (first flex signal). Flips once at
    /// the run end on a perfect run, holds through a bounded window, and books
    /// the evt_signal_perfect funnel line exactly once.&lt;/summary&gt;
    public sealed class GoldenRodView
    {
        public const double HoldSec = 1.20;              // golden banner window (bounded)
        public static string BannerLine =&gt; RodTexts.Active(RodTexts.KeyGoldenBanner);
        public static string CtaLine =&gt; RodTexts.Active(RodTexts.KeyGoldenCta);
        public double FlippedAt = -1.0;                  // run-end stamp (-1 = never flipped)
        public bool Golden;
        public readonly System.Collections.Generic.List&lt;string&gt; Funnel = new System.Collections.Generic.List&lt;st
        bool funnelBooked;
        static readonly Color Gold = Hex(0xFFD54A);      // the golden accent
        static readonly Color Cream = Hex(0xF5EFE0);     // polaroid cream CTA
        readonly Transform root;
        public GoldenRodView(Transform root) { this.root = root; }
        /// &lt;summary&gt;Run-end booking: a perfect run flips the banner on this
        /// very tick and books the funnel line exactly once; anything short
        /// keeps the face dark and the funnel clean.&lt;/summary&gt;
        public bool NoteRun(CastModel m, int hintsUsed)
        {
            Golden = PerfectLaw.IsPerfect(m, hintsUsed);
            if (Golden)
            {
                FlippedAt = m.Now;
                if (!funnelBooked)
                {
                    Funnel.Add("evt_signal_perfect@" + m.Now.ToString("0.00"));
                    funnelBooked = true;
                }
            }
            return Golden;
        }
        /// &lt;summary&gt;Builds the golden rig: banner + CTA. Returns the number of
        /// objects created (frozen footprint: 2).&lt;/summary&gt;
```

```
    public int Rebuild()
    {
        FaceKit.ClearChildren(root);
        int built = 0;
        built += FaceKit.MakeText(root, "golden_banner", BannerLine, new Vector3(0f, 21.0f, -1f), Gold, 34, 0.42f) !
        built += FaceKit.MakeText(root, "golden_cta", CtaLine, new Vector3(0f, 18.6f, -1f), Cream, 22, 0.30f) != nul
        return built;
    }
    /// &lt;summary&gt;Per-step face on the explicit clock: the banner and CTA
    /// hold through the bounded golden window after the flip tick.&lt;/summary&gt;
    public void ApplyAt(double now)
    {
        var banner = FaceKit.FindChild(root, "golden_banner");
        var cta = FaceKit.FindChild(root, "golden_cta");
        if (banner == null || cta == null) return;
        bool live = Golden &amp;&amp; FlippedAt &gt;= 0.0 &amp;&amp; (now - FlippedAt) &lt;= HoldSec;
        banner.SetActive(live);
        cta.SetActive(live);
    }
    static Color Hex(int rgb)
    {
        return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
    }
}
/// &lt;summary&gt;Shared procedural face kit for the signal rigs (the same
/// null-safe pattern every ReelRiot view already uses - gates never break
/// on a missing face).&lt;/summary&gt;
// Public for the Editor evidence rigs (T042-c item 3: the zh frame path
// must drive the REAL production font switch, not a rig-side copy).
public static class FaceKit
{
    static Sprite circle;
    public static GameObject FindChild(Transform root, string childName)
    {
        if (root == null) return null;
        Transform t = root.Find(childName);
        return t == null ? null : t.gameObject;
    }
    public static void ClearChildren(Transform root)
    {
        if (root == null) return;
        for (int i = root.childCount - 1; i &gt;= 0; i--) WaterView.SafeDestroy(root.GetChild(i).gameObject);
    }
    public static TextMesh MakeText(Transform parent, string name, string text, Vector3 localPos, Color color, int f
    {
        if (parent == null) return null;
        var go = new GameObject(name);
        go.transform.SetParent(parent, false);
        go.transform.localPosition = localPos;
        var tm = go.AddComponent&lt;TextMesh&gt;();
```

```
        tm.text = text;
        tm.anchor = TextAnchor.MiddleCenter;
        tm.alignment = TextAlignment.Center;
        tm.fontSize = fontSize;
        tm.characterSize = characterSize;
        tm.color = color;
        Font f = TextFont();
        if (f != null) tm.font = f;
        return tm;
    }
    public static GameObject MakeCircle(Transform parent, string name, Color color, Vector3 localPos, Vector3 scale,
    {
        if (parent == null) return null;
        var go = new GameObject(name);
        go.transform.SetParent(parent, false);
        go.transform.localPosition = localPos;
        go.transform.localScale = scale;
        var sr = go.AddComponent&lt;SpriteRenderer&gt;();
        sr.sprite = Circle;
        sr.color = color;
        sr.sortingOrder = sortingOrder;
        return go;
    }
    static Sprite Circle
    {
        get
        {
            if (circle != null) return circle;
            int size = 32;
            var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
            float r = size * 0.5f;
            for (int y = 0; y &lt; size; y++)
            {
                for (int x = 0; x &lt; size; x++)
                {
                    float dx = x + 0.5f - r;
                    float dy = y + 0.5f - r;
                    bool inside = dx * dx + dy * dy &lt;= r * r;
                    tex.SetPixel(x, y, inside ? Color.white : Color.clear);
                }
            }
            tex.Apply();
            circle = Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
            return circle;
        }
    }
    static Font textFont;
    static Font TextFont()
    {
        if (textFont != null) return textFont;
```

```
            // T037 slice c: the zh face rides the subset pixel CJK font; the
            // default face keeps the built-in font, pixel-identical to the
            // pre-i18n look.
            if (RodTexts.UseZh)
            {
                try { textFont = Resources.Load&lt;Font&gt;("Fonts/rod_font_zh"); }
                catch { textFont = null; }
                if (textFont != null) return textFont;
            }
            try { textFont = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf"); }
            catch { textFont = null; }
            if (textFont == null)
            {
                try { textFont = Resources.GetBuiltinResource&lt;Font&gt;("Arial.ttf"); }
                catch { textFont = null; }
            }
            return textFont;
        }
    }
}
// SittingGoal.cs
// SittingGoal.cs - concurrent catch contracts for one sitting session (G05-T031).
// Every sitting opens 2-3 authored "catch contracts" (triple combo / heavy
// haul / perfect sitting) - the session's micro-arcs riding on top of the cast
// loop. Zero RNG: the contract set derives from the session seed alone (same
// seed -&gt; same contracts), progress books on the resolve tick and a finished
// contract settles on that very beat (same-beat settlement law - the reward
// never lags the action), a rescue voids the perfect-sitting contract, and
// swapping the dead contract out never touches the chain or the earned rare
// points (no-penalty swap). The session keeps its OWN event ledger - the
// classic cast stream stays byte-frozen (replay parity law); the host wires
// OnResolve/OnRescue exactly the way SaveWire's resolve beat is wired.
using System;
using System.Collections.Generic;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;Authored contract shapes (taskbook examples: 3-combo / heavy x2 / zero-rescue perfect sitting).&l
    public enum GoalKind { Combo3 = 0, Heavy2 = 1, Clean3 = 2 }
    /// &lt;summary&gt;Lifecycle of one contract slot: Active plays, Settled paid out, Failed swap-eligible.&lt;/summary
    public enum GoalState { Active = 0, Settled = 1, Failed = 2 }
    /// &lt;summary&gt;One authored contract definition - every number lives here, never at a call site.&lt;/summary&gt;
    public sealed class GoalDef
    {
        public GoalKind kind;
        public string title;      // English UI copy (source-string law)
        public int target;        // authored completion count
        public int rarePoints;    // rare-slot unlock progress granted on settle
    }
    /// &lt;summary&gt;The authored contract pool (zero RNG face - static table).&lt;/summary&gt;
    public static class GoalTable
```

```
{
    public static readonly GoalDef[] Defs =
    {
        new GoalDef { kind = GoalKind.Combo3, title = RodTexts.Active(RodTexts.KeyGoalTitleCombo3), target = 3, rare
        new GoalDef { kind = GoalKind.Heavy2, title = RodTexts.Active(RodTexts.KeyGoalTitleHeavy2), target = 2, rare
        new GoalDef { kind = GoalKind.Clean3, title = RodTexts.Active(RodTexts.KeyGoalTitleClean3), target = 3, rare
    };
}
/// &lt;summary&gt;One live contract slot on the session board.&lt;/summary&gt;
public sealed class GoalContract
{
    public GoalDef Def;
    public GoalState State = GoalState.Active;
    public int Progress;
    public double SettleT = -1.0;   // resolve tick the contract settled on (-1 while open)
}
/// &lt;summary&gt;Session-local ledger beat (mirror of CastEvent - kept separate so the cast stream stays byte-froz
public sealed class GoalEvent
{
    public double t;
    public string type;
    public string detail;
    public GoalEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
}
/// &lt;summary&gt;One sitting's contract board (G05-T031).&lt;/summary&gt;
public sealed class GoalSession
{
    public readonly List&lt;GoalContract&gt; Contracts = new List&lt;GoalContract&gt;();
    public readonly List&lt;GoalEvent&gt; Events = new List&lt;GoalEvent&gt;();
    public int Chain;        // settled contracts this session (a swap never breaks it)
    public int RareEarned;   // rare-slot points settled this session
    public int ChainTier { get { return 1 + Chain / 2; } }   // a cabin upgrade every 2 consecutive settles
    int swapCursor;
    double nowT;             // mirrors the cast clock at each hook beat
    /// &lt;summary&gt;Opens the session: 2 + (seed &amp;amp; 1) concurrent contracts
    /// drawn from the authored table at a seed-stride - pure arithmetic,
    /// zero RNG, the same seed always derives the same board.&lt;/summary&gt;
    public void Begin(int seed)
    {
        int count = 2 + (seed &amp; 1);   // authored 2-3 concurrent band
        for (int i = 0; i &lt; count; i++)
            Contracts.Add(new GoalContract { Def = GoalTable.Defs[(seed + i * 2) % GoalTable.Defs.Length] });
        swapCursor = seed % GoalTable.Defs.Length;
        Log("goal_open", count + " contracts on the board");
    }
    /// &lt;summary&gt;Resolve-beat host hook: books progress on the fight's own
    /// resolve tick and settles any contract that just hit its authored
    /// target on that same tick (never a later frame).&lt;/summary&gt;
    public void OnResolve(CastModel m)
    {
```

```
    if (m == null) return;
    nowT = m.Now;
    bool landed = m.Outcome == CastOutcome.Caught;
    bool heavy = landed &amp;&amp; m.CurrentFight != null &amp;&amp; m.CurrentFight.spec.tensionCap &gt; 0;   //
    for (int i = 0; i &lt; Contracts.Count; i++)
    {
        var c = Contracts[i];
        if (c.State != GoalState.Active) continue;
        switch (c.Def.kind)
        {
            case GoalKind.Combo3:
                c.Progress = landed ? c.Progress + 1 : 0;   // a deslot breaks the streak, never the contract
                break;
            case GoalKind.Heavy2:
                if (heavy) c.Progress++;
                break;
            case GoalKind.Clean3:
                if (landed &amp;&amp; m.Rescues == 0) c.Progress++;
                break;
        }
        if (c.State == GoalState.Active &amp;&amp; c.Progress &gt;= c.Def.target) Settle(i);
    }
}
/// &lt;summary&gt;Rescue-beat host hook: the ad comeback voids the
/// perfect-sitting contract (a rescue can never un-happen) - failed,
/// not punished: swapping the dead slot out is free.&lt;/summary&gt;
public void OnRescue(CastModel m)
{
    if (m == null) return;
    nowT = m.Now;
    for (int i = 0; i &lt; Contracts.Count; i++)
    {
        if (Contracts[i].State != GoalState.Active || Contracts[i].Def.kind != GoalKind.Clean3) continue;
        Contracts[i].State = GoalState.Failed;
        Log("goal_fail", "rescue accepted - the perfect-sitting contract is void, swap it free");
    }
}
void Settle(int index)
{
    var c = Contracts[index];
    c.State = GoalState.Settled;
    c.SettleT = nowT;
    Chain++;
    RareEarned += c.Def.rarePoints;
    Log("goal_settle", c.Def.title + " - done (+" + c.Def.rarePoints + " rare points)");
    Log("chain_tier", "contract chain tier " + ChainTier + " (chain " + Chain + ")");
}
/// &lt;summary&gt;No-penalty swap: a failed contract trades its slot for the
/// next authored def (skipping the dead kind and live duplicates when
/// the table allows it) - the chain and the earned rare points never
```

```
        /// move. Settled and active contracts are not swappable: the board only
        /// ever trades what died.&lt;/summary&gt;
        public bool Swap(int index)
        {
            if (index &lt; 0 || index &gt;= Contracts.Count) return false;
            if (Contracts[index].State != GoalState.Failed) return false;
            GoalKind dead = Contracts[index].Def.kind;
            Contracts[index] = new GoalContract { Def = NextDef(dead) };
            Log("goal_swap", "swapped in: " + Contracts[index].Def.title + " - no penalty, chain " + Chain + " stands");
            return true;
        }
        GoalDef NextDef(GoalKind exclude)
        {
            // pass 1: a def that is neither the dead kind nor a live duplicate;
            // pass 2: the small authored table can run out of fresh kinds, so a
            // duplicate is better than a dead-on-arrival re-issue - still never
            // the excluded kind, still pure cursor arithmetic.
            for (int pass = 0; pass &lt; 2; pass++)
            {
                for (int walk = 0; walk &lt; GoalTable.Defs.Length; walk++)
                {
                    GoalDef def = GoalTable.Defs[swapCursor];
                    swapCursor = (swapCursor + 1) % GoalTable.Defs.Length;
                    if (def.kind == exclude) continue;
                    if (pass == 1) return def;
                    bool dup = false;
                    for (int i = 0; i &lt; Contracts.Count; i++)
                        if (Contracts[i].State == GoalState.Active &amp;&amp; Contracts[i].Def.kind == def.kind) { dup =
                    if (!dup) return def;
                }
            }
            return GoalTable.Defs[swapCursor];   // unreachable with a 3-def table - deterministic fallback
        }
        void Log(string type, string detail) { Events.Add(new GoalEvent(nowT, type, detail)); }
    }
}
// SlowMoFx.cs
// SlowMoFx.cs - so-close slow-mo beat (G05-T025 first-wow signal): a deslot
// while the album sits one slot short (the armed dex itch - the spotlight has
// already locked the missing slot) dips the global clock for a short unscaled
// window - the fish that was ONE cast from completing the set flies off the
// hook in recordable slow motion, and the SO CLOSE tag invites the screen grab
// (zero-server: the phone's own recorder captures, the share card spreads).
// Ported from the G04-T021 collapse beat (ScrewOut SlowMoFx - proven one-pass
// green): the restore rides unscaled time so a frame hitch can never strand
// the game in slow motion, and Advance() lets headless gates tick the restore
// deterministically.
using UnityEngine;
namespace Sy.ReelRiot
{
```

```
    public sealed class SlowMoFx : MonoBehaviour
    {
        public const float Life = 0.9f;      // unscaled seconds of hero time
        public const float Factor = 0.35f;   // global clock dip during the window
        float age;
        TextMesh screenTag;
        /// &lt;summary&gt;Engages the window on the host (idempotent: a chain of
        /// so-close deslots is one slow-mo, not one per beat).&lt;/summary&gt;
        public static SlowMoFx Engage(GameObject hostGo, TextMesh slowMoTag)
        {
            SlowMoFx fx = hostGo.GetComponent&lt;SlowMoFx&gt;();
            if (fx != null) return fx;
            fx = hostGo.AddComponent&lt;SlowMoFx&gt;();
            fx.screenTag = slowMoTag;
            Time.timeScale = Factor;
            return fx;
        }
        void Update() { Advance(Time.unscaledDeltaTime); }
        /// &lt;summary&gt;Pure restore tick (unscaled dt) - gates drive this directly.&lt;/summary&gt;
        public void Advance(float unscaledDt)
        {
            age += unscaledDt;
            if (age &gt;= Life) Restore();
        }
        /// &lt;summary&gt;Ends the window: clock back to 1.0, tag lights out, fx self-removes.&lt;/summary&gt;
        public void Restore()
        {
            Time.timeScale = 1f;
            if (screenTag != null) screenTag.gameObject.SetActive(false);
            if (Application.isPlaying) Destroy(this);
            else DestroyImmediate(this);
        }
        void OnDestroy()
        {
            // Play-mode teardown safety (scene unload / object destroy): never
            // strand a slow clock. Edit-mode gates always close the window
            // through the explicit Restore - a non-ExecuteAlways component
            // receives no editor-phase callbacks, so batchmode gates can never
            // lean on this hook.
            if (Time.timeScale == Factor) Time.timeScale = 1f;
        }
    }
}
// TensionView.cs
// TensionView.cs - procedural placeholder for the tug-of-war rod arc (G05-T006).
// The visual hammer is the bent-rod arc (pitch book: the struggle silhouette):
// every tension beat yanks the rod toward the fish's pull direction - the fish
// fights YOU, not the other way around - holds the bend through the beat, then
// eases back to rest before the next beat lands. Motion reads the model's
// logical clock (never wall time), so headless gates see the exact bend the
```

```
// player sees. X027 wiring rule: "rod"/"rod_tip"/"tension_glow" art stems
// upgrade in place at S2.5; the procedural fallback is null-safe so gates
// never break on missing art.
using UnityEngine;
using Sy.Core;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;Builds and drives the bent-rod arc for the tug-of-war struggle.
    /// Call Rebuild once, then Apply(model) after every model step.&lt;/summary&gt;
    public sealed class TensionView
    {
        public const int RodSegs = 4;            // arc segments above the water edge
        public const float SegBaseY = 15.5f;     // rod butt just above the pond top edge
        public const float SegStepY = 1.2f;
        public const double BeatHoldSec = 0.20;  // bend holds at full peak
        public const double EaseBackSec = 0.30;  // then eases back to rest (done before the next beat)
        public const float BendUnitsPerAmplitude = 2.0f; // amplitude 0.50 -&gt; 1.0-unit tip swing
        static readonly Color RodColor = Hex(0xB98A4A);   // warm placeholder timber
        static readonly Color GlowColor = Hex(0xFF8A3D);  // beat flash - tension reads at a squint
        readonly Transform root;
        static Sprite solid;
        static Sprite circle;
        public TensionView(Transform root) { this.root = root; }
        /// &lt;summary&gt;Builds the rod rig: RodSegs arc segments, the rod tip and
        /// the tension glow. Returns the number of objects created.&lt;/summary&gt;
        public int Rebuild()
        {
            Clear();
            int built = 0;
            for (int k = 0; k &lt; RodSegs; k++)
            {
                Sprite rodArt = ArtRuntime.Get("rod");
                Sprite segFace = rodArt != null ? rodArt : Solid;
                built += Make(
                    segFace,
                    RodColor,
                    "rod_seg_" + k,
                    new Vector3(0f, SegBaseY + k * SegStepY, 0f),
                    WaterView.FitScale(segFace, 0.30f, 1.30f),
                    6) != null ? 1 : 0;
            }
            Sprite tipArt = ArtRuntime.Get("rod_tip");
            Sprite tipFace = tipArt != null ? tipArt : Solid;
            built += Make(
                tipFace,
                RodColor,
                "rod_tip",
                new Vector3(0f, SegBaseY + RodSegs * SegStepY, 0f),
                WaterView.FitScale(tipFace, 0.55f, 0.55f),
                7) != null ? 1 : 0;
```

```
        Sprite glowArt = ArtRuntime.Get("tension_glow");
        Sprite glowFace = glowArt != null ? glowArt : Circle;
        built += Make(
            glowFace,
            GlowColor,
            "tension_glow",
            new Vector3(0f, SegBaseY + RodSegs * SegStepY, 0f),
            WaterView.FitScale(glowFace, 1.7f, 1.7f),
            8) != null ? 1 : 0;
        return built;
    }
    /// &lt;summary&gt;Per-step update: bends the rod arc along the fight's latest
    /// tension beat and flashes the glow while the bend lives. No fight or a
    /// settled beat leaves the rod at rest (all offsets zero, glow off).&lt;/summary&gt;
    public void Apply(CastModel m)
    {
        if (root == null) return;
        var tip = FindChild("rod_tip");
        if (tip == null) return;
        TensionBeat beat = LatestBeat(m);
        float offset = beat == null ? 0f : TipOffsetForBeat(beat, m.Now);
        for (int k = 0; k &lt; RodSegs; k++)
        {
            var seg = FindChild("rod_seg_" + k);
            if (seg != null)
            {
                seg.transform.localPosition = new Vector3(
                    offset * SegCurveWeight(k), SegBaseY + k * SegStepY, 0f);
            }
        }
        tip.transform.localPosition = new Vector3(
            offset * SegCurveWeight(RodSegs), SegBaseY + RodSegs * SegStepY, 0f);
        var glow = FindChild("tension_glow");
        if (glow != null)
        {
            glow.transform.localPosition = tip.transform.localPosition;
            glow.SetActive(offset != 0f);
        }
    }
    /// &lt;summary&gt;Latest beat of the current fight, null when idle.&lt;/summary&gt;
    public static TensionBeat LatestBeat(CastModel m)
    {
        if (m == null || m.CurrentFight == null || m.CurrentFight.Beats.Count == 0) return null;
        return m.CurrentFight.Beats[m.CurrentFight.Beats.Count - 1];
    }
    /// &lt;summary&gt;Beat envelope by age: full peak through BeatHoldSec, linear
    /// ease-back to zero by BeatHoldSec + EaseBackSec (never reaches the
    /// next pulse - the fastest cadence is 0.50s), zero beyond.&lt;/summary&gt;
    public static double BeatEnvelope(double age)
    {
```

```
    if (age &lt;= BeatHoldSec) return 1.0;
    if (age &lt;= BeatHoldSec + EaseBackSec) return 1.0 - (age - BeatHoldSec) / EaseBackSec;
    return 0.0;
}
/// &lt;summary&gt;Tip swing for a beat at logical time now: direction * amplitude
/// * scale * envelope - the exact number the gate re-derives.&lt;/summary&gt;
public static float TipOffsetForBeat(TensionBeat beat, double now)
{
    return (float)(beat.direction * beat.amplitude * BendUnitsPerAmplitude
        * BeatEnvelope(now - beat.t));
}
/// &lt;summary&gt;Quadratic bend curve weight for arc element k (segments
/// 0..RodSegs-1, tip = index RodSegs): the rod bends progressively -
/// butt barely moves, tip swings hardest (gate-assertable arc shape).&lt;/summary&gt;
public static float SegCurveWeight(int k)
{
    return (k + 1) * (k + 1) / (float)(RodSegs * RodSegs);
}
/// &lt;summary&gt;Looks up a built child object by name. Null when absent.&lt;/summary&gt;
public GameObject FindChild(string childName)
{
    if (root == null) return null;
    Transform t = root.Find(childName);
    return t == null ? null : t.gameObject;
}
public void Clear()
{
    if (root == null) return;
    for (int i = root.childCount - 1; i &gt;= 0; i--) WaterView.SafeDestroy(root.GetChild(i).gameObject);
}
static Sprite Solid
{
    get { if (solid == null) solid = MakeSprite(4, false); return solid; }
}
static Sprite Circle
{
    get { if (circle == null) circle = MakeSprite(32, true); return circle; }
}
static Sprite MakeSprite(int size, bool round)
{
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    float r = size * 0.5f;
    for (int y = 0; y &lt; size; y++)
    {
        for (int x = 0; x &lt; size; x++)
        {
            float dx = x + 0.5f - r;
            float dy = y + 0.5f - r;
            bool inside = !round || dx * dx + dy * dy &lt;= r * r;
            tex.SetPixel(x, y, inside ? Color.white : Color.clear);
```

```
                }
            }
            tex.Apply();
            return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
        }
        GameObject Make(Sprite sprite, Color color, string name, Vector3 localPos, Vector3 scale, int sortingOrder)
        {
            if (root == null) return null;
            var go = new GameObject(name);
            go.transform.SetParent(root, false);
            go.transform.localPosition = localPos;
            go.transform.localScale = scale;
            var sr = go.AddComponent&lt;SpriteRenderer&gt;();
            sr.sprite = sprite;
            sr.color = color;
            sr.sortingOrder = sortingOrder;
            return go;
        }
        static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
    }
}
// WaterModel.cs
// WaterModel.cs - pure C# water data layer for ReelRiot (G05-T004).
// Byte-parity port of the E4 graybox dense-water contract (GrayboxSim): field
// dimensions carry the Stage3 perf promise (1200-fish dense water, 32-segment
// rod line, 1200-cell ripple grid - resolved at 0.029ms/cast in the graybox),
// the fish spec table is the frozen feel ladder (three weight classes cycling
// by cast index, slip limit 2), and the absurd-catch codex is the blind-box
// comedy engine: twelve categories in fixed rotation order - zero RNG anywhere,
// outcomes stay pure tug-timing skill. No UnityEngine dependency: headless
// checks drive everything deterministically.
using System;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;One tug-of-war opponent: struggle pulses and the slip budget (E4-frozen feel ladder).&lt;/summary
    public sealed class FishSpec
    {
        public int pulses;        // struggle pulls in the tug-of-war
        public double pulseDur;   // sim seconds per pull
        public double sinkDur;    // lure sink time before the bite
        public int slipLimit;     // missed tugs before the hook slips (deslot)
        public int sprintPulse;   // deep-water heavyweight: the fish-sprint beat index, -1 when the fight is classic
        public int sprintPulse2; // late-deep escalation (F1 grip law): second sprint beat index, -1 when single-sprint
        public int tensionCap;    // deep-water heavyweight: held-beat tension table capacity, 0 when classic
        public bool deepFight;    // true once the sitting has entered deep water (cast &gt;= DeepWaterFromCast)
        public int weightClass;   // T033: 1/2/3 - the class the records book keys the heaviest catch on (set by the cla
    }
```

```
/// &lt;summary&gt;Fish spec table: three weight classes on an authored per-round
/// schedule (T029 F5 pressure ramp). Rounds 0-5 keep the frozen E4 cycle
/// (cast 0 light, cast 2 heavyweight, cast 3 back to light - byte-parity
/// with the graybox FishSpecFor where the frozen checks read it), then the
/// pond deepens: rounds 6-7 trade the light slot for a second mid, rounds
/// 8-9 for a second heavyweight - the per-round pulse average rises
/// 4.00 -&gt; 4.33 -&gt; 4.67 so a sitting always ends at higher pressure than
/// it opened (F5-pressure-curve law). Zero RNG: the schedule is authored,
/// never rolled.&lt;/summary&gt;
public static class FishTable
{
    public const int DeepWaterFromCast = 3;   // sprint + tension table wake up here
    public const int LateDeepFromCast = 21;   // F1 grip law: from here the heavyweight runs twice - holding either
    static readonly int[] Schedule =
    {
        1, 2, 3,   1, 2, 3,   1, 2, 3,   1, 2, 3,   // rounds 0-3: frozen E4 cycle
        1, 2, 3,   1, 2, 3,                          // rounds 4-5: still the classic cycle
        2, 2, 3,   2, 2, 3,                          // rounds 6-7: light fades out
        2, 3, 3,   2, 3, 3,                          // rounds 8-9: deep water
    };
    public static FishSpec SpecFor(int castIndex)
    {
        int weightClass = Schedule[castIndex % Schedule.Length];
        return ByClass(weightClass, castIndex &gt;= DeepWaterFromCast, castIndex);
    }
    // Class-spec builders - the single source every feel literal lives in:
    // the classic schedule and the T030 band lanes both derive through
    // them, so a spec value can never fork across two tables.
    public static FishSpec LightSpec(bool deep)
    {
        return new FishSpec { weightClass = 1, pulses = 3, pulseDur = 0.50, sinkDur = 0.65, slipLimit = 2, sprintPul
    }
    public static FishSpec MidSpec(bool deep)
    {
        return new FishSpec { weightClass = 2, pulses = 4, pulseDur = 0.55, sinkDur = 0.85, slipLimit = 2, sprintPul
    }
    public static FishSpec HeavySpec(bool deep, int castIndex)
    {
        return new FishSpec { weightClass = 3, pulses = 5, pulseDur = 0.70, sinkDur = 1.05, slipLimit = 2, sprintPul
    }
    public static FishSpec ByClass(int weightClass, bool deep, int castIndex)
    {
        if (weightClass == 1) return LightSpec(deep);
        if (weightClass == 2) return MidSpec(deep);
        return HeavySpec(deep, castIndex);
    }
}
/// &lt;summary&gt;T030 pre-cast lane choice: the player's agency over what the
/// next cast fishes. None keeps the frozen classic schedule (byte-parity
/// law - every existing gate reads it), Shallow is the light fun lane,
```

```
/// Deep is the heavy risk lane.&lt;/summary&gt;
public enum BandChoice { None = 0, Shallow = 1, Deep = 2 }
/// &lt;summary&gt;Band lanes (G05-T030): authored per-band weight schedules and
/// category tables. Shallow = light/mid classes, classic easy fights,
/// light fun props. Deep = mid/heavy classes with the deep-fight law
/// always awake - sprint beats and the tension table run from cast 0 -
/// and the heavy-haul props, so the deep lane's deslot risk is provably
/// higher while a release-on-telegraph master still lands it. Zero RNG:
/// both tables are authored and rotate by cast index - the same band plus
/// the same plans always derive the identical sitting (conditional
/// determinism).&lt;/summary&gt;
public static class BandSpecs
{
    static readonly int[] ShallowSchedule = { 1, 1, 2, 1, 2, 1, 1, 2, 2, 1 };
    static readonly int[] DeepSchedule = { 3, 2, 3, 3, 2, 3, 3, 3, 2, 3 };
    // Per-band category tables (indices into AbsurdCodex.Items): the
    // shallow lane hands out the light fun props, the deep lane the heavy
    // hauls - the odds face reads straight off these authored tables.
    public static readonly int[] ShallowItems = { 1, 3, 6, 8, 9, 10 };   // rubber_duck, love_letter, rubber_chicken
    public static readonly int[] DeepItems = { 0, 2, 4, 5, 7, 11 };      // rice_cooker, old_boot, traffic_cone, gar
    public static FishSpec SpecFor(BandChoice band, int castIndex)
    {
        if (band == BandChoice.Deep)
        {
            int weightClass = DeepSchedule[castIndex % DeepSchedule.Length];
            return FishTable.ByClass(weightClass, true, castIndex);
        }
        int shallowClass = ShallowSchedule[castIndex % ShallowSchedule.Length];
        return FishTable.ByClass(shallowClass, false, castIndex);
    }
    public static int ItemIndexFor(BandChoice band, int bandCatchIndex)
    {
        int[] table = band == BandChoice.Deep ? DeepItems : ShallowItems;
        return table[((bandCatchIndex % table.Length) + table.Length) % table.Length];
    }
}
/// &lt;summary&gt;Dense water body (E4 Stage3 perf contract): the fish field, the
/// rod line and the ripple grid as raw float arrays - the render layer only
/// reads state. Deterministic fill: two builds are byte-identical, no
/// randomness face exists anywhere in the data layer.&lt;/summary&gt;
public sealed class WaterBodyData
{
    public const int FishCount = 1200;     // E4: dense-water casts resolve with 1200 live fish
    public const int LineSegments = 32;
    public const int RippleCells = 1200;
    public const float Width = 40f;
    public const float Height = 30f;
    public readonly float[] FishX = new float[FishCount];
    public readonly float[] FishY = new float[FishCount];
    public readonly float[] LineX = new float[LineSegments];
```

```
public readonly float[] LineY = new float[LineSegments];
public readonly float[] Ripple = new float[RippleCells];
/// &lt;summary&gt;Deterministic dense fill (graybox parity): fish scattered
/// across the 40x30 pond, the rod line hanging from mid-surface, ripples
/// pre-lapping. Seeds flow through the canonical seed functions so the
/// fill and every gate read the exact same floats (single-source law).&lt;/summary&gt;
public void BuildDense()
{
    for (int i = 0; i &lt; FishCount; i++)
    {
        FishX[i] = (i * 0.37f) % Width;
        FishY[i] = (i * 0.61f) % Height;
        Ripple[i] = RippleSeed(i);
    }
    for (int i = 0; i &lt; LineSegments; i++)
    {
        LineX[i] = 20f;
        LineY[i] = LineSeedY(i);
    }
}
/// &lt;summary&gt;Canonical ripple seed (graybox formula): the gate recomputes
/// through this same function, so a BuildDense formula drift can never
/// hide behind cross-assembly float re-derivation.&lt;/summary&gt;
public static float RippleSeed(int i)
{
    return ((i * 7) % 13) * 0.05f;
}
/// &lt;summary&gt;Canonical rod-line seed (graybox formula).&lt;/summary&gt;
public static float LineSeedY(int i)
{
    return 15f + i * 0.25f;
}
/// &lt;summary&gt;School density query: fish whose y sits inside [y0, y1).
/// Feeds the view band tint (the fish crowd reads at a squint) and the
/// later bite-rate tuning - pure data, no side effects.&lt;/summary&gt;
public int FishInBand(float y0, float y1)
{
    int n = 0;
    for (int i = 0; i &lt; FishCount; i++)
        if (FishY[i] &gt;= y0 &amp;&amp; FishY[i] &lt; y1) n++;
    return n;
}
/// &lt;summary&gt;Cast-line bite pulse (T005): the bite ripple is real water
/// state the view reads - fixed energy into the canonical anchor cell,
/// then one graybox relaxation pass. Deterministic: two pulses from
/// the same fill land on the same floats.&lt;/summary&gt;
public void PulseRipple()
{
    Ripple[0] = Ripple[0] + 0.35f;
    for (int c = 0; c &lt; RippleCells; c++)
```

```
        {
            Ripple[c] = (Ripple[c] + Ripple[(c * 31 + 7) % RippleCells]) * 0.5f;
        }
    }
}
/// &lt;summary&gt;Absurd-catch codex: the twelve-category blind-box catalog in
/// fixed rotation order. The rice cooker leads - the first-laugh blind box
/// (red line 11 hook). Zero gambling vocabulary ships with the data: the
/// banned token list rides along so the hygiene assert stays checker-side
/// (compound wheel forms - the graybox attempt2 checker fix ports with it).&lt;/summary&gt;
public sealed class AbsurdCodex
{
    public static readonly string[] Items =
    {
        "rice_cooker", "rubber_duck", "old_boot", "love_letter",
        "traffic_cone", "garden_gnome", "rubber_chicken", "bicycle_wheel",
        "pink_flamingo", "tv_remote", "eggplant_plush", "rusty_anchor"
    };
    public static readonly string[] GamblingTokens =
    { "coin", "jackpot", "slot", "casino", "prize_wheel", "spin_wheel", "lucky_wheel", "777", "vegas", "bet" };
    /// &lt;summary&gt;Zero-RNG law (E3 condition 3): stays 0 forever - resolution is pure skill.&lt;/summary&gt;
    public const int RngCalls = 0;
    public const bool HasRewardWheel = false;
    public int Order;                           // fixed rotation cursor
    public readonly bool[] Seen = new bool[Items.Length];
    /// &lt;summary&gt;Records one landed catch: the fixed rotation hands out the
    /// next catalog item and marks it seen. Returns the item id.&lt;/summary&gt;
    public string RecordCatch()
    {
        int idx = Order % Items.Length;
        Order++;
        Seen[idx] = true;
        return Items[idx];
    }
    /// &lt;summary&gt;Records one landed catch straight at an authored table
    /// index (T030 band lane): marks the catalog slot seen and returns the
    /// item id without touching the classic rotation cursor - a band
    /// catch can never shift the classic stream (isolation law).&lt;/summary&gt;
    public string RecordCatchAt(int idx)
    {
        int i = ((idx % Items.Length) + Items.Length) % Items.Length;
        Seen[i] = true;
        return Items[i];
    }
    public int DistinctCount()
    {
        int n = 0;
        for (int i = 0; i &lt; Seen.Length; i++) if (Seen[i]) n++;
        return n;
    }
```

```
    }
}
// WaterView.cs
// WaterView.cs - procedural placeholder rendering for the water body (G05-T004).
// X027 wiring rule: use ArtCatalog art when a stem ships, otherwise fall back
// to procedural color blocks; both paths are null-safe so headless gates never
// break on missing art. The density band tint reads at a squint - a crowded
// band reads darker, the fish crowd is the player's bite-rate hint - and the
// codex strip shows the twelve absurd categories with seen slots lit.
// S2-T015: RebuildExpansion adds the 24-slot page's expansion layer below the
// base strip - the LiveOps landing row (unlocked) or the expand CTA chip (not
// yet) - without touching a single base-strip anchor.
using UnityEngine;
using Sy.Core;
namespace Sy.ReelRiot
{
    /// &lt;summary&gt;Builds placeholder visuals for a dense water body plus the absurd
    /// codex strip. Call Rebuild after any state change.&lt;/summary&gt;
    public sealed class WaterView
    {
        public const int BandCount = 6;    // horizontal density bands across the pond
        public const float UnseenAlpha = 0.35f;
        // S2-T015 expansion layer: the 24-slot dex page. The landing row is
        // the LiveOps rotating-category spots (T016+ fills them); until the
        // ad unlock the row's only face is the accent expand CTA chip.
        public const int LiveOpsSlots = 12;          // landing-row slots behind the unlock
        public const float LiveOpsRowGapY = 1.10f;   // the landing row rides this far below the base strip
        public const float LandingAlpha = 0.22f;     // hollow landing spot - dimmer than any seen/unseen base slot
        public const string ExpandCtaName = "dex_expand_cta";
        // S2-T016 rotation faces: the day variant lights the LiveOps landing
        // row (featured categories carry the day accent) and underlines the
        // hot density band (the bite-rate hint). Additive layer only - the
        // base surface, the six density bands and the twelve codex slots
        // stay byte-identical whatever the day brings.
        public const string HotBandName = "rot_hot_band";
        public const float FeaturedAlpha = 0.85f;   // featured landing slot - lit by the day accent
        public const float HotBandAlpha = 0.85f;    // the hot-band emphasis strip
        // S2-T032 rare-stamp faces: the authored skill stamps ride the
        // expansion page as an additive layer above the landing row - locked
        // faces show the visible condition with the exact shortfall, earned
        // faces show the gold stamp title. The hollow landing spots, the
        // featured tints and the CTA chip never move.
        public const float RareFaceOffsetY = 0.55f;   // the stamp caption rides this far above its landing spot
        static readonly Color SurfaceColor = Hex(0x14424E);   // deep teal pond
        static readonly Color BandBase = Hex(0x1E6E7E);       // school tint
        static readonly Color ExpandCtaColor = Hex(0xE8833A); // buoy warm orange - the expand face
        static readonly Color CtaCappedColor = Hex(0x6B6B6B); // budget spent - the honest grey
        static readonly Color LandingColor = Hex(0x7A8A8F);   // neutral landing spot (LiveOps placeholder)
        static readonly Color RareLockedColor = Hex(0x8FA3B5);   // T032: steel - the locked condition face
        static readonly Color RareUnlockedColor = Hex(0xFFD54A);   // T032: gold - the earned stamp face
```

```
const float BandAlphaMin = 0.20f;
const float BandAlphaSpan = 0.55f;
readonly Transform root;
static Sprite solid;
static Sprite circle;
public WaterView(Transform root) { this.root = root; }
/// &lt;summary&gt;Rebuilds all visuals: the water surface, BandCount density
/// bands tinted by the school count, and the twelve-slot codex strip.
/// Returns the number of objects created.&lt;/summary&gt;
public int Rebuild(WaterBodyData water, AbsurdCodex codex)
{
    Clear();
    int built = 0;
    Sprite surfaceArt = ArtRuntime.Get("water_surface");
    Sprite surfaceFace = surfaceArt != null ? surfaceArt : Solid;
    built += Make(
        surfaceFace,
        SurfaceColor,
        "water_surface",
        Vector3.zero,
        FitScale(surfaceFace, WaterBodyData.Width, WaterBodyData.Height),
        0) != null ? 1 : 0;
    float bandH = WaterBodyData.Height / BandCount;
    int busiest = 0;
    var counts = new int[BandCount];
    for (int b = 0; b &lt; BandCount; b++)
    {
        // data space is y in [0, Height): band b covers [b*bandH, (b+1)*bandH)
        counts[b] = water.FishInBand(b * bandH, (b + 1) * bandH);
        if (counts[b] &gt; busiest) busiest = counts[b];
    }
    for (int b = 0; b &lt; BandCount; b++)
    {
        float t = busiest &gt; 0 ? (float)counts[b] / busiest : 0f;
        Color c = BandBase;
        c.a = BandAlphaMin + BandAlphaSpan * t;
        built += Make(
            Solid,
            c,
            "fish_band_" + b,
            new Vector3(0f, -WaterBodyData.Height * 0.5f + (b + 0.5f) * bandH, 0f),
            new Vector3(WaterBodyData.Width, bandH, 1f),
            2) != null ? 1 : 0;
    }
    for (int i = 0; i &lt; AbsurdCodex.Items.Length; i++)
    {
        string item = AbsurdCodex.Items[i];
        Sprite itemArt = ArtRuntime.Get("catch_" + item);
        Sprite slotFace = itemArt != null ? itemArt : Circle;
        Color c = ItemColor(i);
```

```
        c.a = codex != null &amp;amp;&amp;amp; codex.Seen[i] ? 1f : UnseenAlpha;
        built += Make(
            slotFace,
            c,
            "codex_" + item,
            SlotPos(i),
            FitScale(slotFace, 0.9f, 0.9f),
            4) != null ? 1 : 0;
    }
    return built;
}
/// &lt;summary&gt;Deterministic twelve-hue palette for the codex slots (one hue
/// per item - the squint glance reads the set, gate-assertable determinism).&lt;/summary&gt;
public static Color ItemColor(int index)
    =&gt; Color.HSVToRGB((index * 0.0833f) % 1f, 0.55f, 0.9f);
/// &lt;summary&gt;Fit-scale law (T022, G04 X152 fourth-law paradigm): the
/// authored template must occupy exactly the world rectangle the
/// procedural placeholder covered at this call site, whatever its pixel
/// size or import PPU - the art swap stays geometry-inert. The procedural
/// sprites are one-unit squares, so a null art face keeps today's raw
/// scale byte-for-byte; a shipped template divides the target by its
/// sprite bounds. Degenerate bounds fall back to the raw scale.&lt;/summary&gt;
public static Vector3 FitScale(Sprite sprite, float worldW, float worldH)
{
    if (sprite == null) return new Vector3(worldW, worldH, 1f);
    var b = sprite.bounds.size;
    if (b.x &lt;= 0f || b.y &lt;= 0f) return new Vector3(worldW, worldH, 1f);
    return new Vector3(worldW / b.x, worldH / b.y, 1f);
}
public const float SlotGapX = 1.35f;   // codex strip slot spacing
/// &lt;summary&gt;World position of dex slot i - the single source for the
/// strip layout (the T014 dex-hook spotlight locks this exact spot; no
/// cross-file formula re-derivation can drift from the built strip).
/// S2-T015: indices at or past the base twelve land on the LiveOps
/// row below - the base-row formula is byte-identical to the
/// pre-expansion strip, so the hook, the share cards and every
/// existing gate keep their exact anchors.&lt;/summary&gt;
public static Vector3 SlotPos(int index)
{
    int baseCount = AbsurdCodex.Items.Length;
    if (index &lt; baseCount)
    {
        return new Vector3(
            (index - (baseCount - 1) * 0.5f) * SlotGapX,
            -WaterBodyData.Height * 0.5f - 1.1f,
            0f);
    }
    int li = index - baseCount;
    return new Vector3(
        (li - (LiveOpsSlots - 1) * 0.5f) * SlotGapX,
```

```
        -WaterBodyData.Height * 0.5f - 1.1f - LiveOpsRowGapY,
        0f);
}
/// &lt;summary&gt;Anchor of the expand CTA chip - the landing-row center,
/// single-sourced off SlotPos (the row the unlock reveals).&lt;/summary&gt;
public static Vector3 ExpandCtaPos
{
    get
    {
        Vector3 p = SlotPos(AbsurdCodex.Items.Length);
        p.x = 0f;
        return p;
    }
}
/// &lt;summary&gt;Landing-row slot object name - the check and the future
/// LiveOps host address the row through this single name law.&lt;/summary&gt;
public static string LiveOpsSlotName(int landingIndex) =&gt; "dex_liveops_" + landingIndex;
/// &lt;summary&gt;The 24-slot dex page (S2-T015): the expansion layer rides
/// below the base strip and never touches it. Not expanded: exactly
/// one accent expand CTA chip on the landing row - the persistent IAA
/// face (budget tint = ApplyExpandBudget). Expanded: twelve hollow
/// landing spots awaiting the LiveOps rotating categories (T016+).
/// Returns the number of objects created; call after Rebuild.&lt;/summary&gt;
public int RebuildExpansion(bool expanded)
{
    ClearExpansion();
    if (root == null) return 0;
    if (!expanded)
    {
        return Make(Circle, ExpandCtaColor, ExpandCtaName, ExpandCtaPos, new Vector3(2.2f, 0.55f, 1f), 4) != nul
    }
    int built = 0;
    for (int i = 0; i &lt; LiveOpsSlots; i++)
    {
        Color c = LandingColor;
        c.a = LandingAlpha;
        built += Make(
            Circle,
            c,
            LiveOpsSlotName(i),
            SlotPos(AbsurdCodex.Items.Length + i),
            new Vector3(0.9f, 0.9f, 1f),
            4) != null ? 1 : 0;
    }
    return built;
}
/// &lt;summary&gt;Destroys only the expansion-layer children (the base strip
/// survives every expansion re-render; the rare-stamp faces reset with
/// the page they ride on).&lt;/summary&gt;
public void ClearExpansion()
```

```
{
    var cta = FindChild(ExpandCtaName);
    if (cta != null) SafeDestroy(cta);
    for (int i = 0; i &lt; LiveOpsSlots; i++)
    {
        var s = FindChild(LiveOpsSlotName(i));
        if (s != null) SafeDestroy(s);
    }
    ClearRare();
}
/// &lt;summary&gt;Rare-stamp face object name (T032) - the check and the
/// future stamp host address the faces through this single name law.&lt;/summary&gt;
public static string RareFaceName(int landingIndex) =&gt; "rare_face_" + landingIndex;
/// &lt;summary&gt;Rare-stamp face layer (G05-T032): renders the four
/// authored skill stamps onto the expansion page - a locked face shows
/// the visible condition with the exact shortfall ("12 MORE FLAWLESS
/// FIGHTS"), an earned face shows the gold stamp title on the very
/// beat the tally crossed. Additive and idempotent: the landing spots,
/// the featured tints and the CTA chip never move. Null-safe: a null
/// gate builds nothing (headless gates never break on a missing face).
/// Returns the number of faces created.&lt;/summary&gt;
public int ApplyRareSlots(RareSlotGate gate)
{
    ClearRare();
    if (root == null || gate == null) return 0;
    int built = 0;
    for (int i = 0; i &lt; RareSlotTable.Defs.Length; i++)
    {
        var def = RareSlotTable.Defs[i];
        bool earned = gate.IsUnlocked(i);
        string copy = earned ? def.title : gate.LockedFace(i);
        Vector3 pos = SlotPos(AbsurdCodex.Items.Length + def.landingIndex);
        pos.y += RareFaceOffsetY;
        pos.z = -1f;
        built += FaceKit.MakeText(
            root,
            RareFaceName(def.landingIndex),
            copy,
            pos,
            earned ? RareUnlockedColor : RareLockedColor,
            16,
            0.16f) != null ? 1 : 0;
    }
    return built;
}
/// &lt;summary&gt;Destroys only the rare-stamp face layer (the landing spots
/// and the base strip survive).&lt;/summary&gt;
public void ClearRare()
{
    for (int i = 0; i &lt; RareSlotTable.Defs.Length; i++)
```

```
    {
        var face = FindChild(RareFaceName(RareSlotTable.Defs[i].landingIndex));
        if (face != null) SafeDestroy(face);
    }
}
/// &lt;summary&gt;Budget tint for the expand CTA (the rescue CTA honest-face
/// law): accent while the category has budget left, the honest grey
/// when the 3/day is spent - the tint is the visible signal, the tap
/// chain refuses capped taps before the ad either way.&lt;/summary&gt;
public void ApplyExpandBudget(bool budgetOk)
{
    var cta = FindChild(ExpandCtaName);
    if (cta == null) return;
    var sr = cta.GetComponent&lt;SpriteRenderer&gt;();
    if (sr == null) return;
    Color c = budgetOk ? ExpandCtaColor : CtaCappedColor;
    c.a = 1f;
    sr.color = c;
}
/// &lt;summary&gt;Day-variant face (S2-T016): one accent strip across the
/// day's hot density band (the bite-rate hint) and the LiveOps
/// landing-row tint - featured categories carry the day accent while
/// the hollow rest keep the placeholder face. Call after
/// Rebuild/RebuildExpansion; additive and idempotent (the rotation
/// never touches one base face - the frozen laws hold by design).
/// Returns the number of rotation faces applied.&lt;/summary&gt;
public int ApplyRotation(WaterBodyData water, PondVariant v)
{
    ClearRotation();
    if (root == null || v == null) return 0;
    int applied = 0;
    Color accent = Hex(v.accentHex);
    if (water != null)
    {
        // hot-band emphasis strip: rides the built band's center -
        // the check proves it sits on the very band object Rebuild
        // placed (no cross-file formula re-derivation can drift).
        int hot = PondRotation.HotBand(water, v);
        float bandH = WaterBodyData.Height / BandCount;
        Color c = accent;
        c.a = HotBandAlpha;
        applied += Make(
            Solid,
            c,
            HotBandName,
            new Vector3(0f, -WaterBodyData.Height * 0.5f + (hot + 0.5f) * bandH, 0f),
            new Vector3(WaterBodyData.Width, bandH * 0.22f, 1f),
            3) != null ? 1 : 0;
    }
    // landing-row tint: landing slot i mirrors category i - the
```

```
    // day's featured set lights with the accent, the hollow rest
    // keep the LandingAlpha placeholder face.
    for (int i = 0; i &lt; LiveOpsSlots &amp;&amp; i &lt; AbsurdCodex.Items.Length; i++)
    {
        var slot = FindChild(LiveOpsSlotName(i));
        var sr = slot != null ? slot.GetComponent&lt;SpriteRenderer&gt;() : null;
        if (sr == null) continue;
        if (PondRotation.IsFeatured(v, i))
        {
            Color f = accent;
            f.a = FeaturedAlpha;
            sr.color = f;
            applied++;
        }
    }
    return applied;
}
/// &lt;summary&gt;Destroys only the rotation's own band strip (the landing
/// tints re-render with the next RebuildExpansion pass).&lt;/summary&gt;
public void ClearRotation()
{
    var marker = FindChild(HotBandName);
    if (marker != null) SafeDestroy(marker);
}
// G05-T021 bite-bobber layer: the authored float rides the ripple
// anchor - the visible face of the bite pulse. Additive to every
// frozen face: the base strip, the landing row, the rare stamps and
// the rotation bands keep their exact anchors.
public const string BiteBobberName = "bite_bobber";
/// &lt;summary&gt;World position of the bite-ripple anchor (T021): the
/// 1200-cell ripple grid tiles the 40x30 body as 40 columns x 30
/// rows of unit cells, and PulseRipple lands the bite energy in the
/// canonical anchor cell (index 0) - the float sits on that cell's
/// center, so the visible bite face and the pulsed water state share
/// one single-sourced anchor (no cross-file drift).&lt;/summary&gt;
public static Vector3 BiteBobberPos =&gt; new Vector3(
    0.5f - WaterBodyData.Width * 0.5f,
    0.5f - WaterBodyData.Height * 0.5f,
    0f);
/// &lt;summary&gt;Bite float layer (G05-T021): shows the float bobber while
/// a fight is on, at the exact cell the bite pulse energizes. Art
/// law: the authored float_bobber stem when it ships, the buoy-warm
/// procedural float otherwise (X027 null-safe fallback - headless
/// gates never break on missing art). Additive and idempotent: call
/// after Rebuild; ClearBiteBobber removes only this layer. Returns 1
/// when the float is placed, 0 when hidden.&lt;/summary&gt;
public int ApplyBiteBobber(bool fighting)
{
    return ApplyBiteBobber(fighting, false);
}
```

```
/// &lt;summary&gt;Bite float layer, festival face included (slice c-2):
/// the classic call keeps the exact pre-festival face (authored
/// float_bobber template white-tinted, buoy-orange procedural
/// fallback - zero drift); inside the festival the float goes
/// pumpkin (authored festival_bobber stem when it ships, pumpkin
/// procedural otherwise) and rides above the night wash so the
/// bite signal pops on the dark pond.&lt;/summary&gt;
public int ApplyBiteBobber(bool fighting, bool festival)
{
    ClearBiteBobber();
    if (root == null || !fighting) return 0;
    Sprite bobberArt = festival
        ? ArtRuntime.Get("festival_bobber")
        : ArtRuntime.Get("float_bobber");
    Sprite bobberFace = bobberArt != null ? bobberArt : Circle;
    Color c = bobberArt != null
        ? Color.white
        : Hex(festival ? PumpkinBobberHex : BuoyBobberHex);
    return Make(
        bobberFace,
        c,
        BiteBobberName,
        BiteBobberPos,
        FitScale(bobberFace, 0.8f, 0.8f),
        festival ? 6 : 3) != null ? 1 : 0;
}
/// &lt;summary&gt;Destroys only the bite-bobber layer (every other face
/// survives).&lt;/summary&gt;
public void ClearBiteBobber()
{
    var bobber = FindChild(BiteBobberName);
    if (bobber != null) SafeDestroy(bobber);
}
// G05 festival skin (slice c-2): the Halloween window washes the
// pond in one additive night layer - the squint glance reads
// "something is different on the water today". The single-sourced
// consts are the gate's frozen truth (the check derives its
// expected tint from the very same values, never a re-typed copy).
// ---- Festival dex trio (slice b-3) ----
/// &lt;summary&gt;Festival dex-trio slot object name (slice b-3) - the
/// check and the festival host address the trio through this single
/// name law.&lt;/summary&gt;
public static string FestivalSlotName(int index) =&gt; "festival_slot_" + index;
/// &lt;summary&gt;World position of festival dex slot i - the trio rides
/// its own row below the LiveOps landing row, centered on the
/// three-slot span. Single-sourced like SlotPos: the check locks
/// this exact spot, so no other face can drift from the built
/// trio.&lt;/summary&gt;
public static Vector3 FestivalSlotPos(int index)
{
```

```
        return new Vector3(
            (index - (HalloweenEvent.LimitedSlotCount - 1) * 0.5f) * SlotGapX,
            -WaterBodyData.Height * 0.5f - 1.1f - LiveOpsRowGapY * 2f,
            0f);
    }
    /// &lt;summary&gt;Festival dex trio face (slice b-3): renders the three
    /// limited seasonal slots during the open window - a lit slot marks
    /// the festival fish the player has landed (its dex entry), a dark
    /// slot is the fish still out there. Additive and idempotent like
    /// the night wash: the rebuilt base strip, the expansion page and
    /// the rare stamps never move; the closed window builds nothing
    /// (rollback = closed, never a wipe - the trio returns with next
    /// year's window carrying the persisted mask). Null-safe: a null
    /// seen array builds nothing (headless gates never break).
    /// Returns the number of slots created.&lt;/summary&gt;
    public int ApplyFestivalDex(bool open, bool[] limitedSeen)
    {
        ClearFestivalDex();
        if (root == null || !open || limitedSeen == null) return 0;
        int built = 0;
        for (int i = 0; i &lt; HalloweenEvent.LimitedSlotCount; i++)
        {
            Sprite slotArt = ArtRuntime.Get("catch_" + HalloweenEvent.LimitedItems[i]);
            Sprite slotFace = slotArt != null ? slotArt : Circle;
            Color c = ItemColor(AbsurdCodex.Items.Length + i);
            c.a = limitedSeen.Length &gt; i &amp;&amp; limitedSeen[i] ? 1f : UnseenAlpha;
            built += Make(
                slotFace,
                c,
                FestivalSlotName(i),
                FestivalSlotPos(i),
                FitScale(slotFace, 0.9f, 0.9f),
                4) != null ? 1 : 0;
        }
        return built;
    }
    /// &lt;summary&gt;Destroys only the festival dex-trio children (the base
    /// strip, the expansion page and the rare stamps survive).&lt;/summary&gt;
    public void ClearFestivalDex()
    {
        for (int i = 0; i &lt; HalloweenEvent.LimitedSlotCount; i++)
        {
            var s = FindChild(FestivalSlotName(i));
            if (s != null) SafeDestroy(s);
        }
    }
    public const string FestivalNightName = "festival_night";
    public const int FestivalNightHex = 0x241A48;    // deep violet night wash
    public const float FestivalNightAlpha = 0.45f;   // the water stays readable under the night
    public const int PumpkinBobberHex = 0xFF7A1A;    // festival float fallback - pumpkin orange
```

```
public const int BuoyBobberHex = 0xE8833A;      // classic float fallback - buoy warm orange
/// &lt;summary&gt;Festival night wash (slice c-2): one full-body violet
/// layer over the pond while the window is open. The host feeds
/// the window verdict (the view never reads the clock - model
/// purity); the checks feed pinned booleans. Additive and
/// idempotent like every festival face: the base strip, the
/// bands, the rotation, the rare stamps and the codex keep their
/// exact anchors. Returns 1 when the night is placed, 0 when
/// closed.&lt;/summary&gt;
public int ApplyFestivalNight(bool open)
{
    ClearFestivalNight();
    if (root == null || !open) return 0;
    Color c = Hex(FestivalNightHex);
    c.a = FestivalNightAlpha;
    return Make(
        Solid,
        c,
        FestivalNightName,
        Vector3.zero,
        FitScale(null, WaterBodyData.Width, WaterBodyData.Height),
        5) != null ? 1 : 0;
}
/// &lt;summary&gt;Destroys only the festival night layer (every other
/// face survives).&lt;/summary&gt;
public void ClearFestivalNight()
{
    var night = FindChild(FestivalNightName);
    if (night != null) SafeDestroy(night);
}
/// &lt;summary&gt;Density band alpha law: the busiest band gets the max tint,
/// emptier bands fade proportionally (gate-assertable determinism).&lt;/summary&gt;
public static float BandAlpha(int count, int busiest)
    =&gt; BandAlphaMin + BandAlphaSpan * (busiest &gt; 0 ? (float)count / busiest : 0f);
/// &lt;summary&gt;Looks up a built child object by name. Null when absent.&lt;/summary&gt;
public GameObject FindChild(string childName)
{
    if (root == null) return null;
    Transform t = root.Find(childName);
    return t == null ? null : t.gameObject;
}
/// &lt;summary&gt;Edit-mode-aware destroy (pit#202), shared by every view/fx
/// clear path the headless VideoSeqCheck rebuild drives: plain Destroy is
/// deferred and in edit mode only emits a warning stack per call, so
/// destroy immediately there.&lt;/summary&gt;
internal static void SafeDestroy(Object target)
{
    if (target == null) return;
    if (Application.isPlaying) Object.Destroy(target);
    else Object.DestroyImmediate(target);
```

```
        }
        public void Clear()
        {
            if (root == null) return;
            for (int i = root.childCount - 1; i &gt;= 0; i--) SafeDestroy(root.GetChild(i).gameObject);
        }
        static Sprite Solid
        {
            get { if (solid == null) solid = MakeSprite(4, false); return solid; }
        }
        static Sprite Circle
        {
            get { if (circle == null) circle = MakeSprite(32, true); return circle; }
        }
        static Sprite MakeSprite(int size, bool round)
        {
            var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
            float r = size * 0.5f;
            for (int y = 0; y &lt; size; y++)
            {
                for (int x = 0; x &lt; size; x++)
                {
                    float dx = x + 0.5f - r;
                    float dy = y + 0.5f - r;
                    bool inside = !round || dx * dx + dy * dy &lt;= r * r;
                    tex.SetPixel(x, y, inside ? Color.white : Color.clear);
                }
            }
            tex.Apply();
            return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
        }
        GameObject Make(Sprite sprite, Color color, string name, Vector3 localPos, Vector3 scale, int sortingOrder)
        {
            if (root == null) return null;
            var go = new GameObject(name);
            go.transform.SetParent(root, false);
            go.transform.localPosition = localPos;
            go.transform.localScale = scale;
            var sr = go.AddComponent&lt;SpriteRenderer&gt;();
            sr.sprite = sprite;
            sr.color = color;
            sr.sortingOrder = sortingOrder;
            return go;
        }
        static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
    }
}
```